Assignment 3: Constrained Optimization, SVMs

This material, no matter whether in printed or electronic form, may be used for personal and non-commercial educational use only. Any reproduction of this material, no matter whether as a whole or in parts, no matter whether in printed or in electronic form, requires explicit prior acceptance of the authors.

Automatic Testing Guidelines

Automatic unittesting requires you to submit a notebook which contains strictly defined objects. Strictness of definition consists of unified shapes, dtypes, variable names and more.

Within the notebook, we provide detailed instruction which you should follow in order to maximise your final grade.

Name your notebook properly, follow the pattern in the template name:

Assignment_N_NameSurname_matrnumber

1. N - number of assignment
2. NameSurname - your full name where every part of the name starts with a capital letter, no spaces
3. matrnumber - you student number on ID card (with k, potenitially with a leading zero)

Don't add any cells but use the ones provided by us. All cells have a unique ID so that the unit test can find it, so please do not add or remove any cell!

Always make sure that implemented functions have the correct output and given variables contain the correct data type. In the descriptions for every function you can find information on what datatype an output should have and you should stick to that in order to minimize conflicts with the unittest. Don't import any other packages than listed in the cell with the "imports" tag.

Questions are usually multiple choice (except the task description says otherwise) and can be answered by changing the given variables to either "True" or "False". "None" is counted as a wrong answer in any case!

Note: Never use variables you defined in another cell in your functions directly; always pass them to the function as a parameter. In the unitest, they won't be available either. If you want to make sure that everything is executable for the unittest, try executing cells/functions individually (instead of running the whole notebook).

Task 1: Constrained optimization

Consider the following primal (constrained convex optimization) problem:

$$\begin{aligned} \text{Minimize} \quad & f(w_1, w_2) = 2\,(w_1^4 + w_2^4) \\ \text{subject to} \quad & h(w_1, w_2) = 4 - w_1 + w_2 \leq 0 \end{aligned}$$

We try to solve it via two ways. Perform the following tasks:

- Compute the Lagrangian $L(w_1, w_2, \alpha)$.
- Calculate its derivatives with respect to w_1 and w_2 and calculate the zeros of these derivatives (w_1^*, w_2^*).

First way:

- Solve the problem using the KKT conditions.

Second way:

- Write down the dual problem and solve it directly without(!) the help of the KKT-conditions. Hint: You need the derivative of $\mathcal{L} = L(w_1^*, w_2^*, \alpha)$ with respect to α.

For your calculation use the given notation.

Calculation 1.1 (2 Points):

$$L(w_1, w_2, \alpha) = f(w_1, w_2) + \alpha h(w_1, w_2)$$

PLug in the functions $f(w_1, w_2)$, $h(w_1, w_2)$:

$$L(w_1, w_2, \alpha) = 2\,(w_1^4 + w_2^4) + \alpha(4 - w_1 + w_2)$$

Expanding the expression we get:

$$L(w_1, w_2, \alpha) = 2\,w_1^4 + 2\,w_2^4 + 4\alpha - \alpha w_1 + \alpha w_2$$

The derivatives of $L(w_1, w_2, \alpha)$ with respect to w_1 and w_2:

$$\frac{\partial L}{\partial w_1} = 8\,w_1^3 - \alpha$$

$$\frac{\partial L}{\partial w_2} = 8\,w_2^3 + \alpha$$

Calculation 1.2 (5 Points):

Solving with KKT conditions:

Setting both to zero:

$$8\,w_1^3 - \alpha = 0$$

$$8\,w_2^3 + \alpha = 0$$

We get:

$$w_1^* = \left(\frac{\alpha}{8}\right)^{\frac{1}{3}}$$

$$w_2^* = -\left(\frac{\alpha}{8}\right)^{\frac{1}{3}}$$

Plug in w_1^* and w_2^* to the constraint:

$$4 - \frac{\alpha^{\frac{1}{3}}}{2} - \frac{\alpha^{\frac{1}{3}}}{2} \leq 0$$

$$4 - \alpha^{\frac{1}{3}} \leq 0$$

$$\alpha \geq 64$$

Calculation 1.3 (8 Points):

Solving the dual problem:

$$L(w_1^*, w_2^*, \alpha) = 2\,(w_1^4 + w_2^4) + \alpha(4 - w_1 + w_2)$$

$$w_1^4 = \left(\frac{\alpha}{8}\right)^{\frac{4}{3}}$$

$$w_2^4 = \left(\frac{\alpha}{8}\right)^{\frac{4}{3}}$$

First term:

$$2\left(2\left(\frac{\alpha}{8}\right)^{\frac{4}{3}}\right)$$

Second term:

$$\alpha(4 - w_1 + w_2) = \alpha\left(4 - \left(\frac{\alpha}{8}\right)^{\frac{1}{3}} - \left(\frac{\alpha}{8}\right)^{\frac{1}{3}}\right)$$

Final equation:

$$4\left(\frac{\alpha}{8}\right)^{\frac{4}{3}} + \alpha\left(4 - \left(\frac{\alpha}{8}\right)^{\frac{1}{3}} - \left(\frac{\alpha}{8}\right)^{\frac{1}{3}}\right)$$

$$L(w_1^*, w_2^*, \alpha) = 4\left(\frac{\alpha}{8}\right)^{\frac{4}{3}} + 4\alpha - 2\alpha\left(\frac{\alpha}{8}\right)^{\frac{1}{3}}$$

Set $\left(\frac{\alpha}{8}\right)^{\frac{1}{3}} = x$ and simplify:

$$4x^4 + 32x^3 - 16x^4$$

$$-12x^4 + 32x^3$$

Derivative wrt x and set to 0:

$$-48x^3 + 96x^2 = 0$$

We get $x = 2$ and $x = 0$

Check the values in $-12x^4 + 32x^3$, at $x = 2, \alpha = 64$ at $x = 0, \alpha = 0$

Calculation 1.4 (10 Points):

YOUR ANSWER HERE

Suppose we replace in the primal optimization problem of C-SVMs the penalty term $\|\xi\|_1 = \sum_{i=1}^{l} \xi_i$ with $\|\xi\|_2^2 = \sum_{i=1}^{l} \xi_i^2$ (that is we use the quadratic hinge loss instead). Thus, the primal problem we are considering is given as follows (with $C > 0$):

$$\begin{aligned} \text{Minimize} \quad & \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{l} \xi_i^2 \\ \text{subject to} \quad & -\left[y_i\left(\mathbf{w}\cdot\mathbf{x}_i - b\right) - 1 + \xi_i\right] \leq 0 \\ \text{and} \quad & -\xi_i \leq 0 \;\text{ for }\; i = 1, \ldots, l \end{aligned}$$

for all $i = 1, \ldots, l$. Give the associated dual optimization problem by making use of the KKT-Theorem and perform the following tasks:

- Why can the KKT-Theorem be applied?
- Calculate the Lagrange function L.
- Calculate its derivatives (with respect to w_i, b and ξ_i) and compute their zeros.
- Write down the dual function (named $\mathcal{L}$ in the slides) and the corresponding dual problem, depending on $2l$ Lagrange multipliers. (You don't need to solve the dual problem.)
- Simplify the dual problem so that it only depends on l Lagrange variables. Hint: The maximization with respect to λ_i can be done quite easily. (Again: You don't need to solve the dual problem. Just find the simplified form.)

Please provide reasoning and explanations in full sentences.

Calculation 1.5 (4 Points):

From the function we see that:

$$f(w, \xi) = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{l} \xi_i^2$$

$$h_1(w) = \frac{1}{2}\|\mathbf{w}\|^2 = \frac{1}{2}\sum_{i=1}^{l} w_l^2$$

the Hessian matrix of $h_1(w)$:

$$\nabla^2 h_1(w) = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{bmatrix}$$

The Hessian matrix is positive definite, function is convex.

$$h_2(\xi) = C\sum_{i=1}^{l} \xi_i^2$$

the Hessian matrix of $h_2(\xi)$:

$$\nabla^2 h_2(\xi) = \begin{bmatrix} 2C & 0 & 0 & \cdots & 0 \\ 0 & 2C & 0 & \cdots & 0 \\ 0 & 0 & 2C & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 2C \end{bmatrix}$$

Since $C > 0$, Hessian matrix is positive definite, function is convex.

The functions are twice continuously differentiable.

$\exists w^{'}$ with $h_i(w^{'}) < 0$ for all $i = 1, \ldots, l$

Calculation 1.6 (3 Points):

Lagrange:

$$L(w, b, \xi, \lambda, \beta) = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{l} \xi_i^2 + \sum_{i=1}^{l} \lambda_i(-y_i\left(\mathbf{w}\cdot\mathbf{x}_i - b\right) - 1 + \xi_i) + \sum_{i=1}^{l} \beta_i(-$$

Calculation 1.7 (6 Points):

Derivatives of L:

$$\frac{\partial L}{\partial w} = w - \sum_{i=1}^{l} \lambda_i y_i x_i$$

$$\frac{\partial L}{\partial b} = -\sum_{i=1}^{l} \lambda_i y_i$$

$$\frac{\partial L}{\partial \xi} = 2C\xi_i + \lambda_i - \beta_i$$

Setting the derivatives to zero:

$$w = \sum_{i=1}^{l} \lambda_i y_i x_i$$

$$\sum_{i=1}^{l} \lambda_i y_i = 0$$

$$2C\xi_i + \lambda_i - \beta_i = 0$$

$$\xi_i = \frac{\beta_i - \lambda_i}{2C}$$

Calculation 1.8 (4 Points):

The dual function $\mathcal{L}$:

$$\mathcal{L}(w, \xi) = \frac{1}{2}\left(\sum_{i=1}^{l} \lambda_i y_i x_i\right)\left(\sum_{j=1}^{l} \lambda_j y_j x_j\right) + C\sum_{i=1}^{l}\left(\frac{\beta_i - \lambda_i}{2C}\right)^2$$

Calculation 1.9 (8 Points):

Simplify the dual problem:

$$\mathcal{L}(w, \xi) = \frac{1}{2}\left(\sum_{i,j=1}^{l} \lambda_i \lambda_j y_i y_j x_i \cdot x_j\right) + \sum_{i=1}^{l}\left(\frac{\beta_i - \lambda_i}{2C}\right)^2 =$$

$$\frac{1}{2}\left(\sum_{i,j=1}^{l} \lambda_i \lambda_j y_i y_j x_i \cdot x_j\right) + \sum_{i=1}^{l}\frac{(\beta_i - \lambda_i)^2}{4C}$$

Task 2: Introduction to Support Vector Machines

The aim of the following task is to equip you with some intuition concerning the application of different SVMs to an easy data set.
You should also observe how different versions of the SVMs with different hyperparameters react to additional noise. To this end we provided you with a function `plot_data` which is intended to create proper visualizations of important characteristics of linear and nonlinear SVMs, like the decision border and support vectors.
The usual routine for applying SVMs in Python, which is also used here, is given by the following sklearn-package: SVC.

Your first task is to get familiar with the `plot_data` function by applying it to the easy data set `radial_data.csv`. Iterate over different kernels and hyperparameters (consider the documentation for details), i.e. perform the following tasks:

- **Code 2.1**:
  - Plot the dataset using the provided functions. (Just execute the existing code.)
  - Using the sklearn-package mentioned above, write a function `iter_degree` that applies a SVM with a `polynomial kernel` and `C=10` to the data and iterates over the degrees (i.e. hyperparameter). This function should then plot each fitted model alongside the data using the provided function `plot_data`. `iter_degree` must return a list of model parameters (in the form of dictionaries) for each model you initiated and fitted on our data. The output of `get_params()` is a dictionary; see sklearn docs. Utilize the `plot_data` function by passing each fitted model and the data itself in order to visualize the results. You will **get 0 points** if you choose to use your **own visualization function over the provided one**!
- **Question 2.1**:
  - Answer some questions about the resulting plots.
- **Code 2.2**:
  - Now, write a similar function called `iter_C` that applies a SVM with an `rbf kernel` and `degree=3` to the data and iterates over the given C values (i.e. hyperparameter) and plots the fitted models alongside the data. This function should return something similar to `iter_degree`. Again, you will **get 0 points** if you choose to use your **own visualization function over the provided one**!
- **Question 2.2**:
  - Again, answer some questions about your results from the previous task.

In [1]:
```
# Nothing to do here, just run the cell.
import numpy as np
import matplotlib.pyplot as plt
import warnings
from sklearn import svm
warnings.filterwarnings("ignore")
```

In [2]:
```
# Nothing to do here, just run the cell.
def load_data():
    """Function allows to load data from csv"""
    Z = np.genfromtxt('radial_data.csv', delimiter=',')

    return Z[:,:-1], Z[:,-1]

def get_meshgrid(X, resolution):
    """Function creates space/grid. Used for plotting"""
    s = np.max(np.abs(X))*1.05
    ls = np.linspace(-s, s, resolution)
    X1,X2 = np.meshgrid(ls, ls, sparse=False)

    return np.c_[X1.ravel(), X2.ravel()]
```

```python
def plot_data(
    X,
    y,
    model = None,
    plot_borders = True,
    plot_classification = True,
    plot_support_vectors = True,
    plot_size = 7,
    resolution = 500,
    title = 'Data visualization',
    color = ['blue','orange']
):
    """Plot data points and an optional model's decision boundaries and c

    Parameters
    ----------
    X : array-like of shape (n_samples, 2)
        Input data points with two features.

    y : array-like of shape (n_samples,)
        Target labels for the data points. Assumes binary classification
        +1 and -1.

    model : object, optional
        A fitted classification model with `predict`, `decision_function`
        `support_`, `kernel`, and possibly `degree` and `C` attributes if
        should be compatible with scikit-learn's SVM models.

    plot_borders : bool, default=True
        If True, plots the decision boundaries of the model.

    plot_classification : bool, default=True
        If True, plots the model's classification regions.

    plot_support_vectors : bool, default=True
        If True, highlights the support vectors used by the model, if app

    plot_size : int, default=7
        Controls the size of the plot.

    resolution : int, default=500
        The resolution of the mesh grid used to plot decision boundaries

    title : str, default='Data visualization'
        Title for the plot.

    color : list of str, default=['blue', 'orange']
        Colors for the two classes. The first color corresponds to the po
        and the second color corresponds to the negative class (-1).
    """

    if model is not None:#if you want to plot model
        if plot_classification and plot_borders:
            col=2  #if you want to plot model and borders
        else :
            col=1  #if you want to plot model only

        fig,axs = plt.subplots(1,col,figsize=(plot_size*col,plot_size))
```

```
        grid = get_meshgrid(X, resolution)
        V = model.support_vectors_
        mask_sv = model.support_ #np.where(np.isin(X[:,0],V[:,0]))[0]

        kernel = model.kernel
        if kernel == 'poly':
            title = f"kernel: {kernel} - degree: {model.degree} - cost:{m
        elif kernel == 'rbf':
            title = f"kernel: {kernel}"
            if model.gamma != "auto_deprecated" :
                title+= f" - gamma: {model.gamma}"
            title += f" - cost: {model.C}"

        for i in range(col):
            if col>1:
                ax = axs[i]
            else:
                ax = axs
            ax.set_aspect('equal')
            if i==0 and plot_borders:
                ax.set_title('Margins - ' + title,fontsize=plot_size*2)
                ax.set_xlabel("$x_1$")
                ax.set_ylabel("$x_2$")
                borders = model.decision_function(grid)
                mask_pos = borders >= 1
                mask_neg = borders <= -1
                ax.scatter(grid[mask_pos,0], grid[mask_pos,1], c=color[0]
                ax.scatter(grid[mask_neg,0], grid[mask_neg,1], c=color[1]
                ax.scatter(X[mask_sv,0], X[mask_sv,1], c='g',label= str(n
            if plot_classification and (i==1 or not plot_borders):
                ax.set_title('Classification - ' + title,fontsize=plot_si
                ax.set_xlabel("$x_1$")
                ax.set_ylabel("$x_2$")
                classification = model.predict(grid)
                mask_pos = classification > 0
                mask_neg = classification < 0
                ax.scatter(grid[mask_pos,0], grid[mask_pos,1], c=color[0]
                ax.scatter(grid[mask_neg,0], grid[mask_neg,1], c=color[1]
                classification = model.predict(X)
                mask_wrong = classification != y
                ax.scatter(X[mask_wrong,0], X[mask_wrong,1], c='magenta',
            m = y > 0
            ax.scatter(X[m,0], X[m,1], c=color[0],label='class +1',s=10)
            m = np.logical_not(m)
            ax.scatter(X[m,0], X[m,1], c=color[1],label='class -1',s=10)
            ax.legend(loc='lower left', fontsize=plot_size*1.5)

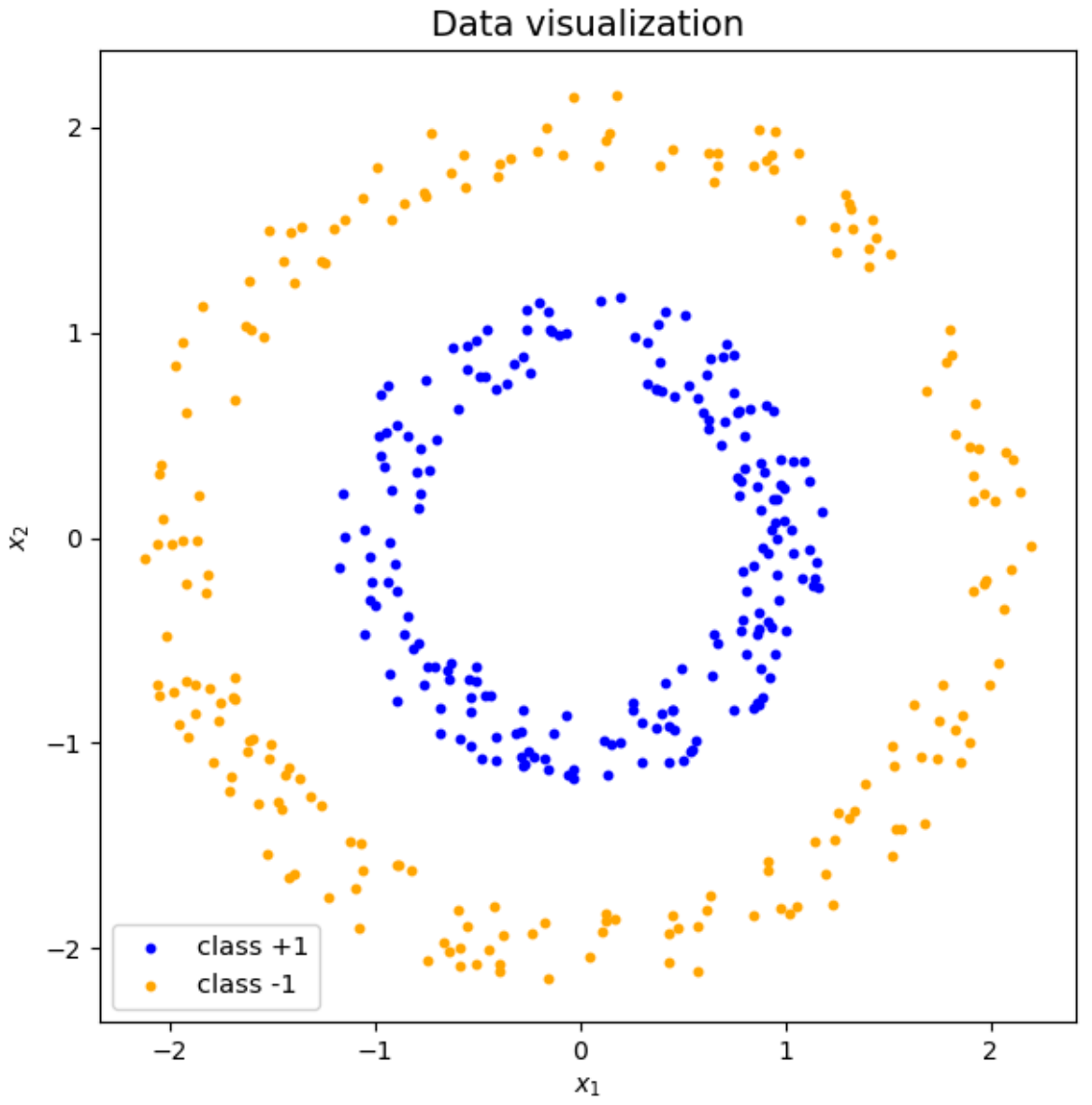
    else:
        fig,axs = plt.subplots(1,1,figsize=(plot_size,plot_size))
        axs.set_aspect('equal')
        m = y > 0
        axs.scatter(X[m,0], X[m,1], c=color[0],label='class +1',s=10)
        m = np.logical_not(m)
        axs.scatter(X[m,0], X[m,1], c=color[1],label='class -1',s=10)
        axs.legend(loc='lower left', fontsize=plot_size*1.5)
        plt.xlabel("$x_1$")
        plt.ylabel("$x_2$")
        plt.title(title, fontsize=plot_size*2)

    plt.show()
```

Code 2.1 (5 Points):

In [3]:
```
# Nothing to do here, just run the cell.
X, y = load_data()
```

In [4]:
```
# Nothing to do here, just run the cell.
plot_data(X, y)
```



In [5]:
```
def iter_degree(degrees: range, X: np.ndarray, y: np.ndarray) -> list[dic
    """Fit an SVM with polynomial kernel and C=10 of varying degrees and

    This function iterates over a specified range of polynomial degrees,
    SVM model with each degree, and plots the decision boundaries. The pa
    of each model are stored in a dictionary and returned in a list.

    Parameters
    ----------
    degrees : range
        Range of integer values representing the polynomial degrees to us
    X : (N, 2) np.ndarray
        Data matrix containing the training data.
```

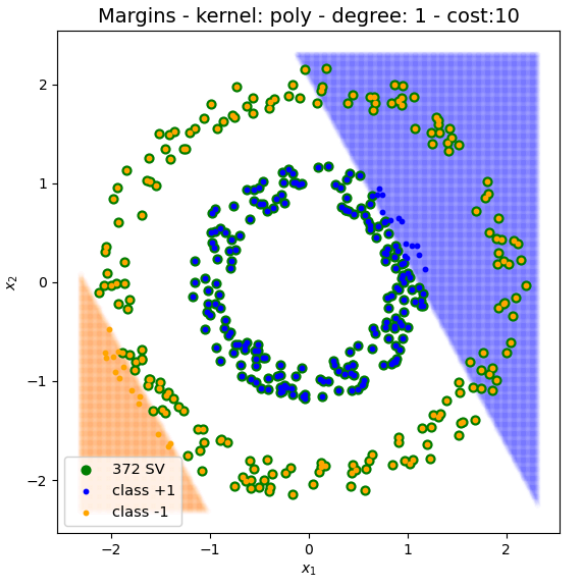
```
    y : (N,) np.ndarray
        Target vector containing the labels.

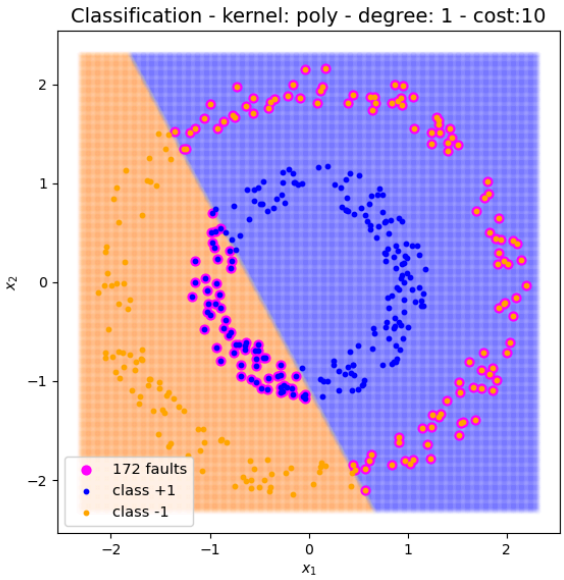
    Returns
    -------
    list of dict
        A list of dictionaries, where each dictionary contains the parame
        with a specific polynomial degree. The length of the list is equa
    """
    # YOUR CODE HERE
    models = []
    for degree in degrees:
        model = svm.SVC(kernel='poly', degree=degree, C=10)
        model.fit(X, y)
        models.append({'model': model, 'degree': degree})

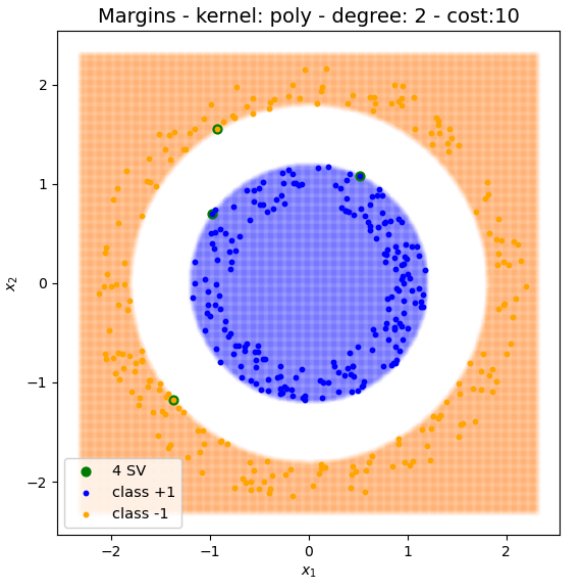
    for model in models:
        plot_data(X, y, model=model['model'])

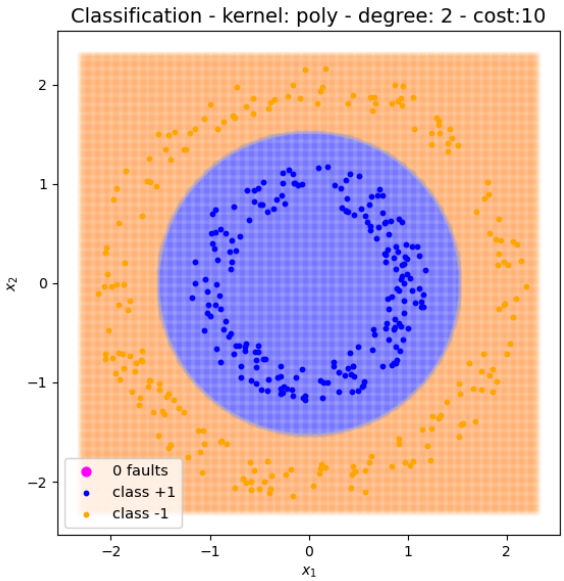
    return models
```

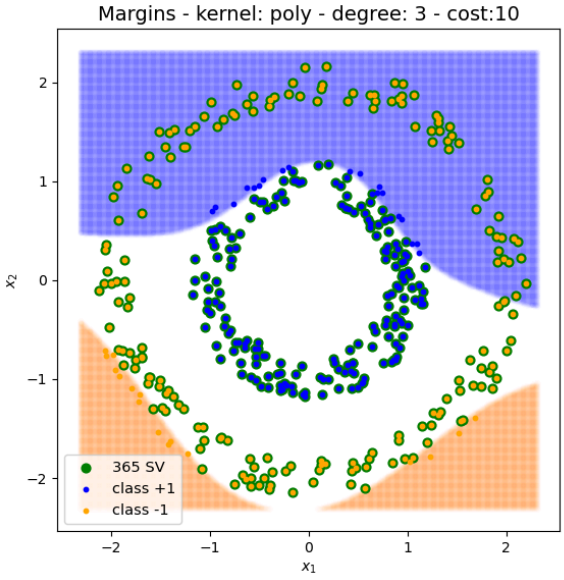
In [6]:
```
# Nothing to do here, just run the cell.
degrees = range(1, 6)
models_degree = iter_degree(degrees, X, y)
```

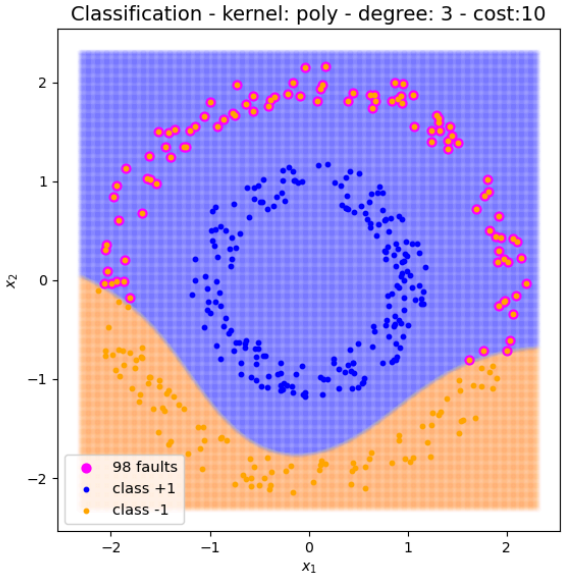


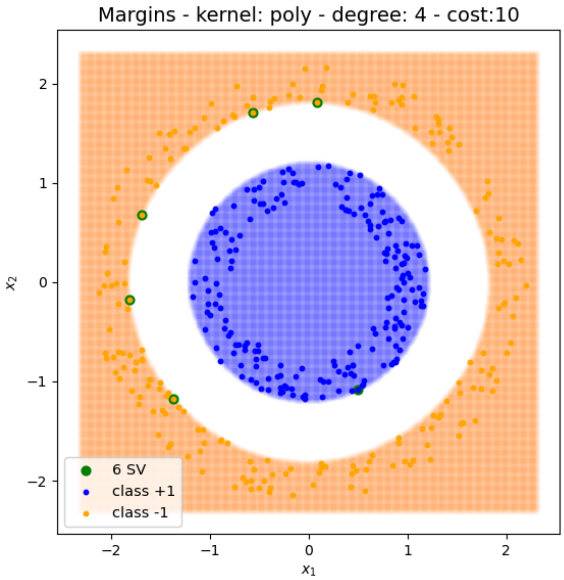


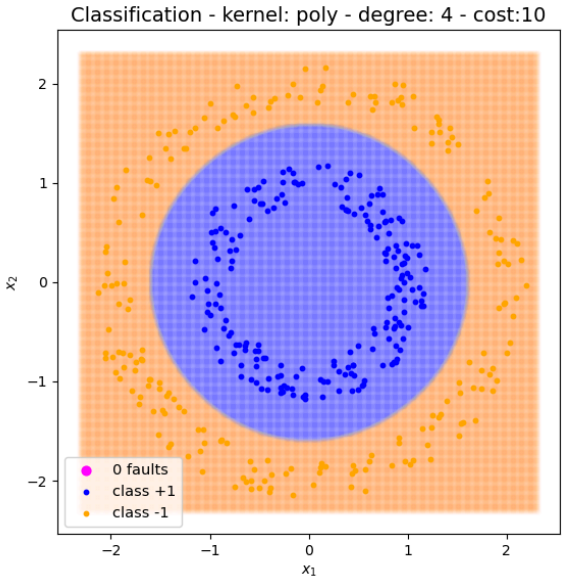


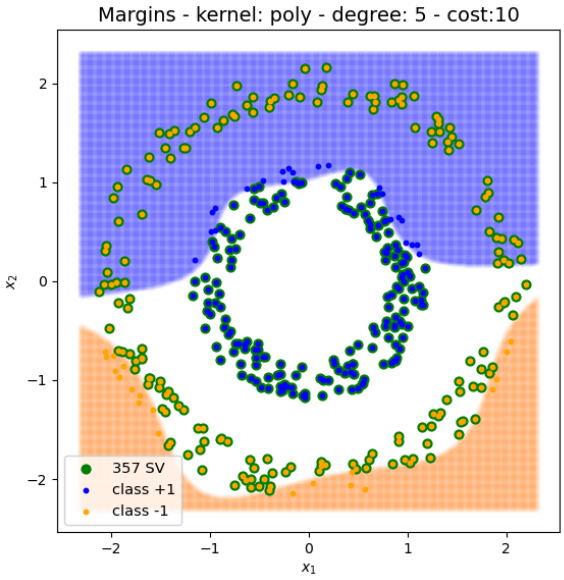


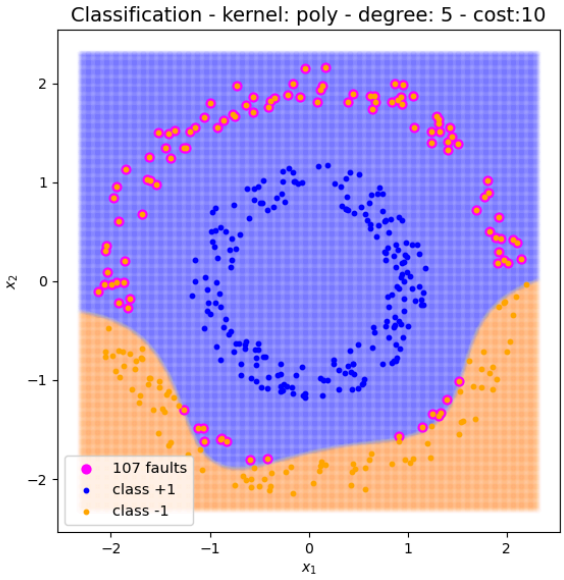












In [7]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(models_degree, list), "The output of iter_degree is not
assert len(models_degree) == len(degrees), "Some models are missing!"
assert all(isinstance(model, dict) for model in models_degree), "At least
```

Question 2.1 (6 Points):

What observations can you make from your plots? (several answers may be correct)

a1_) The SVM with polynomial degree 2 already seems to do quite well on the data set.
b1_) The higher the polynomial degree, the better the classifier.
c1_) A very high number of support vectors seems to be an indicator of a good choice of the kernel.

d1_) There is only small difference between the pictures that were produced by polynomial kernels of even degree.
e1_) For kernels with an odd degree, the number of misclassified samples decreases with an increasing degree.
f1_) For odd degrees, one can see that the model complexity increases with increasing degree.

To answer the question, assign `True` or `False` boolean values to variables in the next cell. For example, if you think that **a1_)** is correct, define a variable `a1_` and set it to `True`, the same applies to **b1_)** and the other options. A non-correctly answered question as well as no answer (i.e. answer "None") yields 0 points for a specific question.

In [8]:
```
# YOUR CODE HERE
a1_ = True
b1_ = False
c1_ = False
d1_ = False
e1_ = False
f1_ = True
```

In [9]:
```
# DO NOT DELETE THIS CELL!
assert a1_ is not None, "Store True/False!"
assert a1_ in [True, False], "Invalid Answer!"
```

In [10]:
```
# DO NOT DELETE THIS CELL!
assert b1_ is not None, "Store True/False!"
assert b1_ in [True, False], "Invalid Answer!"
```

In [11]:
```
# DO NOT DELETE THIS CELL!
assert c1_ is not None, "Store True/False!"
assert c1_ in [True, False], "Invalid Answer!"
```

In [12]:
```
# DO NOT DELETE THIS CELL!
assert d1_ is not None, "Store True/False!"
assert d1_ in [True, False], "Invalid Answer!"
```

In [13]:
```
# DO NOT DELETE THIS CELL!
assert e1_ is not None, "Store True/False!"
assert e1_ in [True, False], "Invalid Answer!"
```

In [14]:
```
# DO NOT DELETE THIS CELL!
assert f1_ is not None, "Store True/False!"
```

```
assert f1_ in [True, False], "Invalid Answer!"
```

Code 2.2 (5 Points):

In [15]:
```
def iter_C(C_values: list[float], X: np.ndarray, y: np.ndarray) -> list[d
    """Iteratively trains SVM models with different values of the regular
    plots the decision boundaries for each model, and returns a list of m

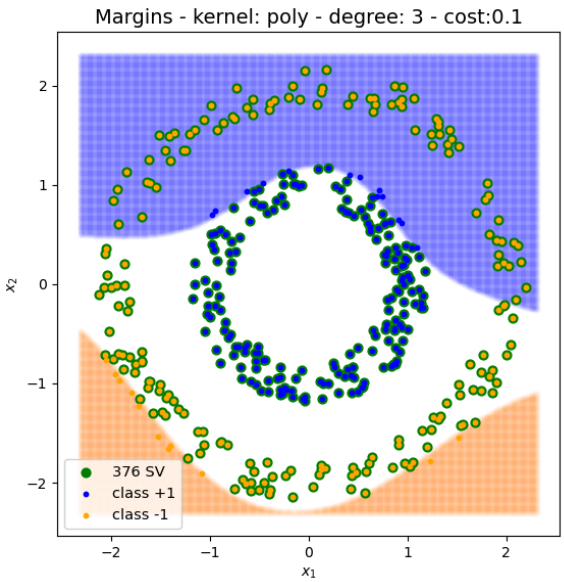
    Parameters
    ----------
    C_values : list of float
        A list of values for the regularization parameter `C` to be used
    X : (N, 2) np.ndarray
        Data matrix containing the training data.
    y : (N,) np.ndarray
        Target vector containing the labels.

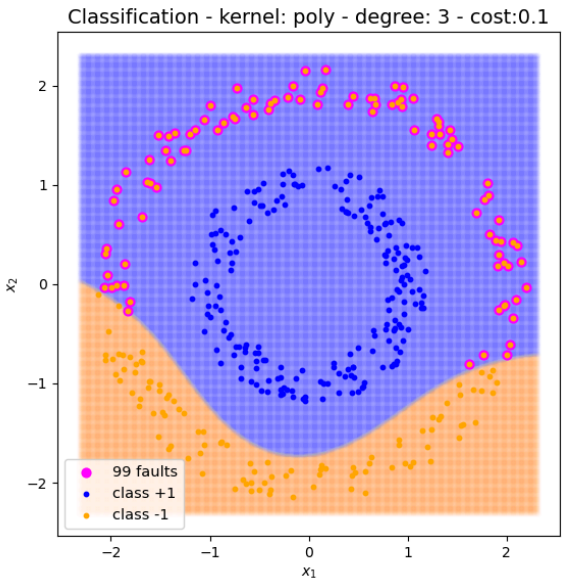
    Returns
    -------
    list of dict
        A list of dictionaries, where each dictionary contains the parame
        for a specific value of `C`.
    """
    # YOUR CODE HERE
    models = []
    for C in C_values:
        model = svm.SVC(kernel='poly', C=C)
        model.fit(X, y)
        models.append({'model': model, 'C': C})

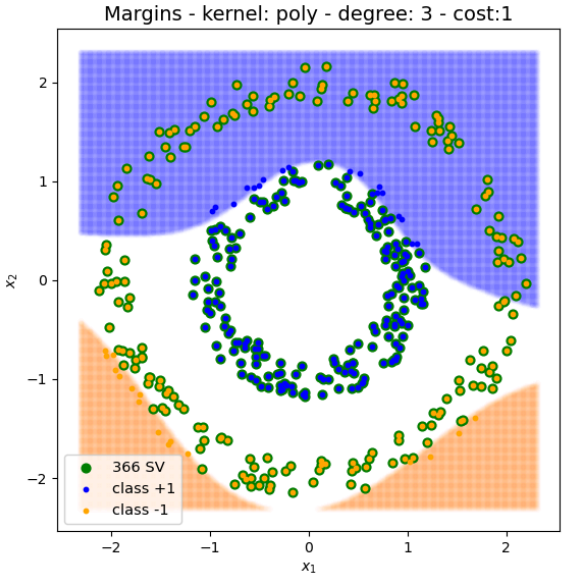
    for model in models:
        plot_data(X, y, model=model['model'])

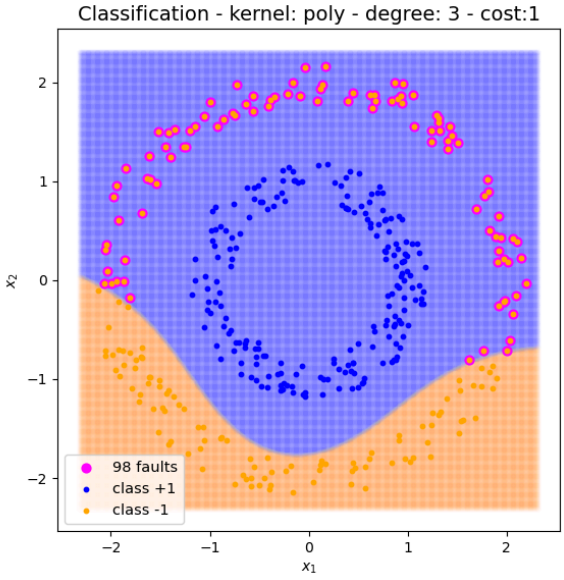
    return models
```

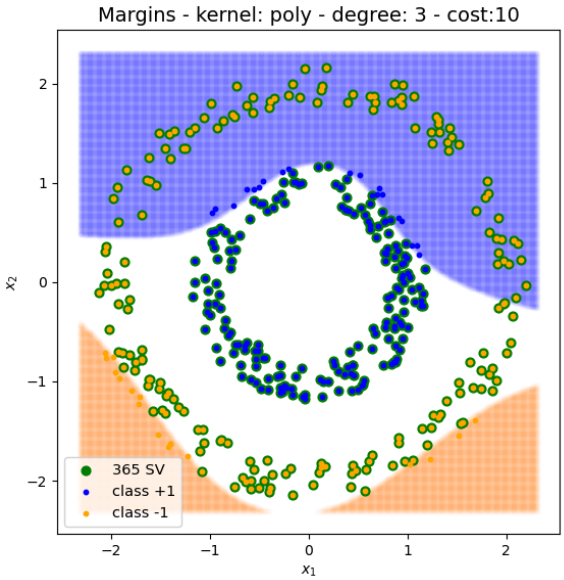
In [16]:
```
# Nothing to do here, just run the cell.
C_values = [0.1, 1, 10, 100, 1000]
models_C = iter_C(C_values, X, y)
```

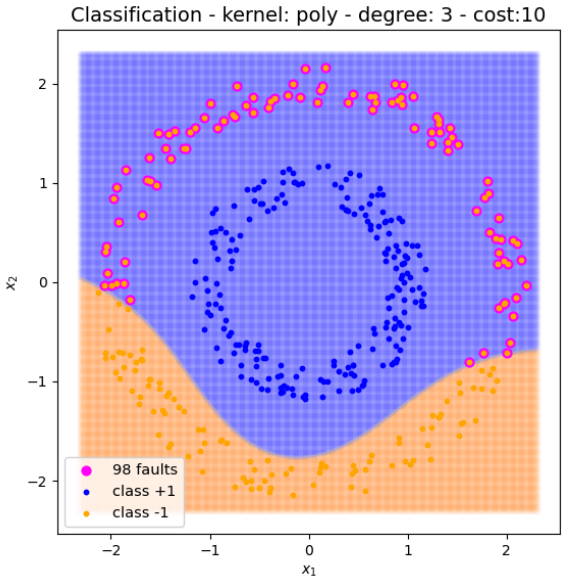


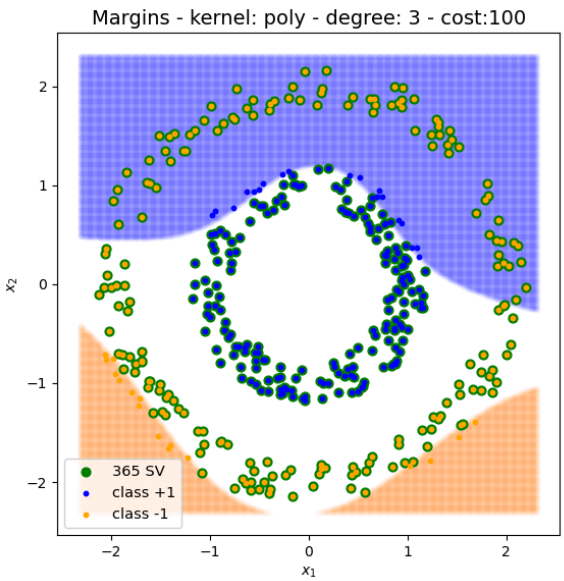


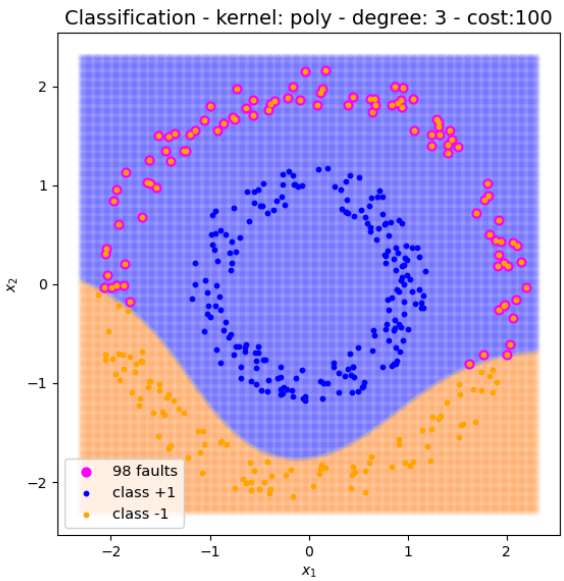

Margins - kernel: poly - degree: 3 - cost:1
366 SV
class +1
class -1
x_1
x_2

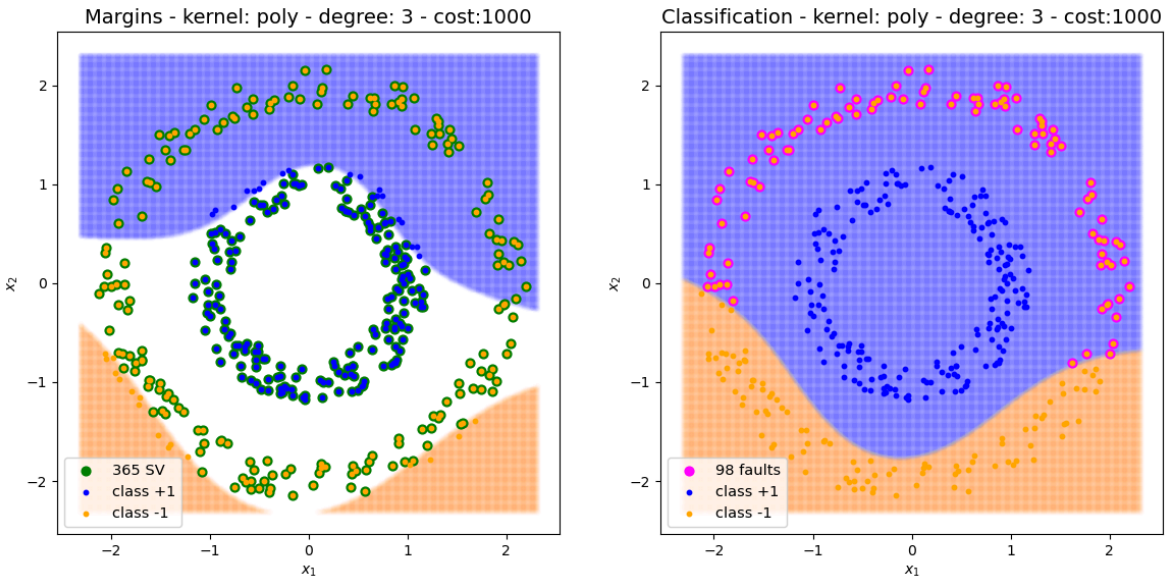

Classification - kernel: poly - degree: 3 - cost:1
98 faults
class +1
class -1
x_1
x_2


Margins - kernel: poly - degree: 3 - cost:10
365 SV
class +1
class -1
x_1
x_2


Classification - kernel: poly - degree: 3 - cost:10
98 faults
class +1
class -1
x_1
x_2


Margins - kernel: poly - degree: 3 - cost:100
365 SV
class +1
class -1
x_1
x_2


Classification - kernel: poly - degree: 3 - cost:100
98 faults
class +1
class -1
x_1
x_2



In [17]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(models_C, list), "The output of iter_degree is not a li
assert len(models_C) == len(C_values), "Some models are missing!"
assert all(isinstance(model, dict) for model in models_C), "At least one
```

Question 2.2 (6 points):

What observations can you make from your plots? Evaluate, if the given statements are true of false (several options may be correct):

a2_) The higher the cost, the more support vectors we have.
b2_) The decision borders don't change drastically with increasing C, only the number of support vectors does.
c2_) The lower the cost, the larger the margin.

To answer the question, assign `True` or `False` boolean values to variables in the next cell. For example, if you think that **a2_)** is correct, define a variable `a2_` and set it to `True`, the same applies to **b2_)** and the other option. A non-correctly answered question as well as no answer (i.e. answer "None") yields 0 points for a specific question.

In [18]:
```
# YOUR CODE HERE
a2_ = False
b2_ = False
c2_ = False
```

In [19]:
```
# DO NOT DELETE THIS CELL!
assert a2_ is not None, "Store True/False!"
assert a2_ in [True, False], "Invalid Answer!"
```

In [20]:
```
# DO NOT DELETE THIS CELL!
assert b2_ is not None, "Store True/False!"
assert b2_ in [True, False], "Invalid Answer!"
```

In [21]:
```
# DO NOT DELETE THIS CELL!
assert c2_ is not None, "Store True/False!"
```

```
assert c2_ in [True, False], "Invalid Answer!"
```

Task 3: SVM with new data

- **Code 3.1**:
  - Use the given function (polar-to-cartesian) `pol2cart()` in order to generate **100 two-dimensional** data points which are **uniformly** distributed across the area of a circle with a radius of `r=0.25`. Label these points with `-1` and add them to the given `X` data matrix and `y` label vector. Note: In polar coordinates, in order to get uniformly distributed samples, you sample the angle uniformly, but be careful with the radius. If you simply sample the radius uniformly, you will not end up with uniformly sampled points within the circle. Hint: Have a look at the formula for the area of a circle and use np.random.rand for sampling points from a uniform distribution.
- **Code 3.2**:
  - Now use an `rbf kernel` (which is the default option for `sklearn.svm.SVC`) and again play around with the parameters to explore the effects on the classification performance by appropriately using the `plot_data` function. Write a function `iter_gamma_C`, analogous to `iter_degree` and `iter_C`. `iter_gamma_C` iterates over different costs C and over different values of $\gamma := 1/(2\sigma^2)$ (compare with the RBF definition in the lecture slides).
- **Question 3.2**:
  - Answer some questions about your plots.

Code 3.1 (7 Points):

In [22]:
```
# Nothing to do here, just run the cell.
def pol2cart(r: np.ndarray, phi: np.ndarray) -> tuple[np.ndarray, np.ndar
    """Function that converts arrays of polar coordinates to arrays of ca
    x = r * np.cos(phi)
    y = r * np.sin(phi)

    return x, y
```

In [ ]:
```
def create_new_data(X: np.ndarray, y: np.ndarray) -> tuple[np.ndarray, np
    """Generates 100 uniformly distributed 2D data points within a circle
    labels them as -1, and appends them to the existing data.

    Parameters
    ----------
    X : (N, 2) np.ndarray
        Original data matrix.
    y : (N,) np.ndarray
        Original label vector.

    Returns
    -------
    tuple of np.ndarray
        - X_new : (N+100, 2) np.ndarray
```

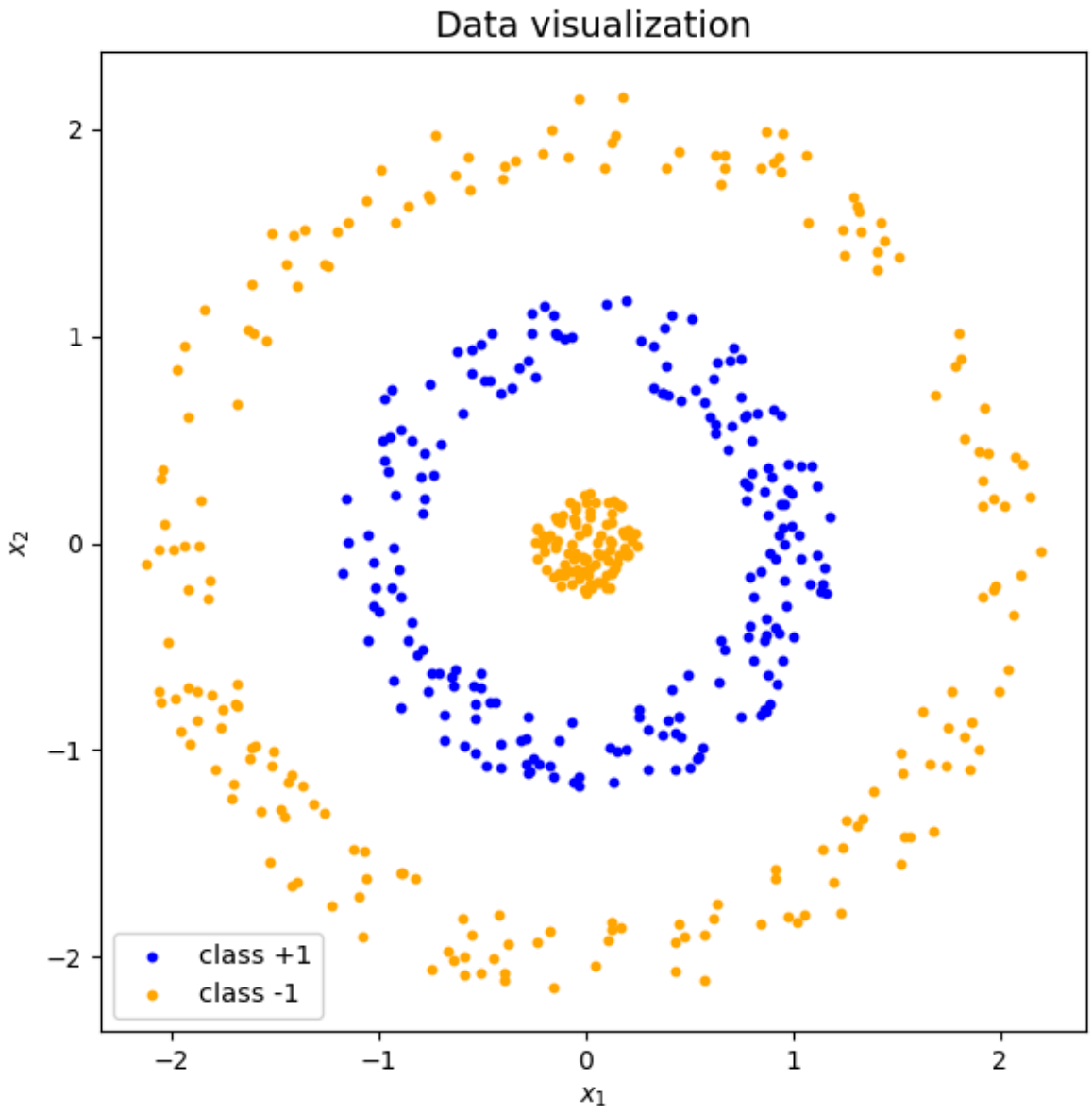
```
            New data matrix.
        - y_new : (N+100,) np.ndarray
            New label vector.
    """

    # YOUR CODE HERE
    rad = np.sqrt(np.random.rand(100)) * 0.25
    ang = np.random.rand(100) * (2 * np.pi)
    x, y_new_points = pol2cart(rad, ang)
    new_points = np.column_stack((x, y_new_points))
    X_new = np.vstack((X, new_points))
    y_new = np.hstack((y, -1 * np.ones(100)))

    return X_new, y_new
```

In [24]:
```
# DO NOT DELETE THIS CELL!
X_new, y_new = create_new_data(X, y)
assert isinstance(X_new, np.ndarray) and isinstance(y_new, np.ndarray), "
assert X_new.shape == (X.shape[0]+100, X.shape[1]), "You should add exact
assert all(y_new[-100:] == -1), "The new samples should have the label -1
assert all(np.sqrt(X_new[-100:, 0]**2 + X_new[-100:, 1]**2) <= 0.25), "So
```

In [25]:
```
# Nothing to do here, just run the cell.
plot_data(X_new, y_new)
```



Code 3.2 (6 Points):

In [26]:
```
def iter_gamma_C(gamma_range: list[float], C_values: list[float], X: np.n
    """Fits an SVM model for each combination of gamma and C values and p

    Parameters
    ----------
    gamma_range : list of float
        List of gamma values for the SVM model.
    C_values : list of float
        List of regularization parameter (C) values for the SVM model.
    X : (N, 2) np.ndarray
        Data matrix.
    y : (N,) np.ndarray
        Label vector.

    Returns
    -------
    list of dict
        A list of dictionaries containing the parameters of each fitted S
    """
    # YOUR CODE HERE
```

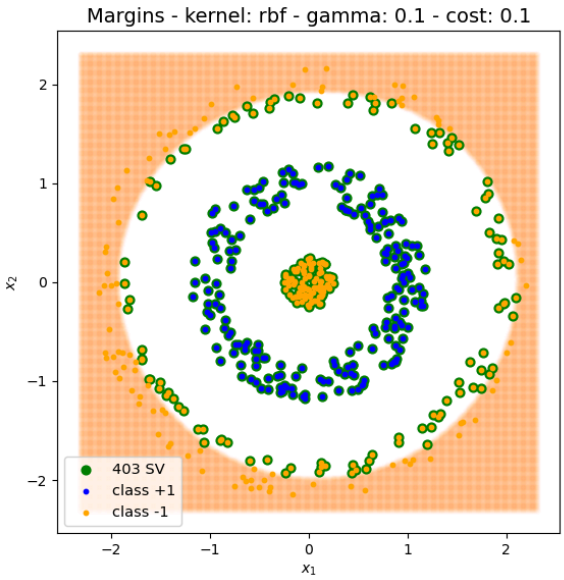
```
    models = []
    for gamma in gamma_range:
        for C in C_values:
            model = svm.SVC(kernel='rbf', gamma=gamma, C=C)
            model.fit(X, y)
            models.append({'model': model, 'gamma': gamma, 'C': C})

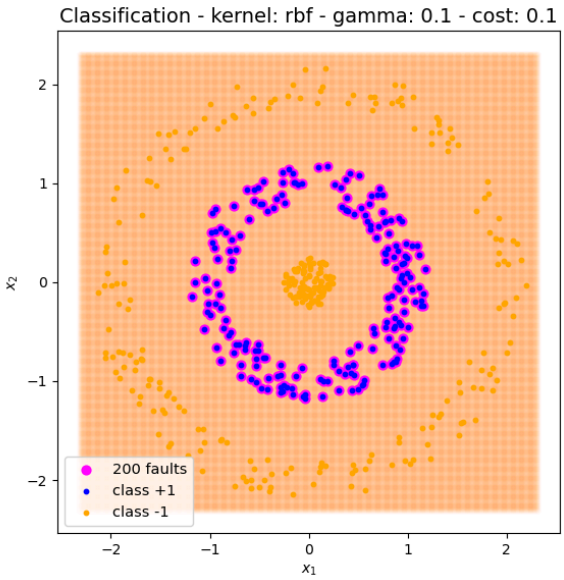
    for model in models:
        plot_data(X, y, model=model['model'])

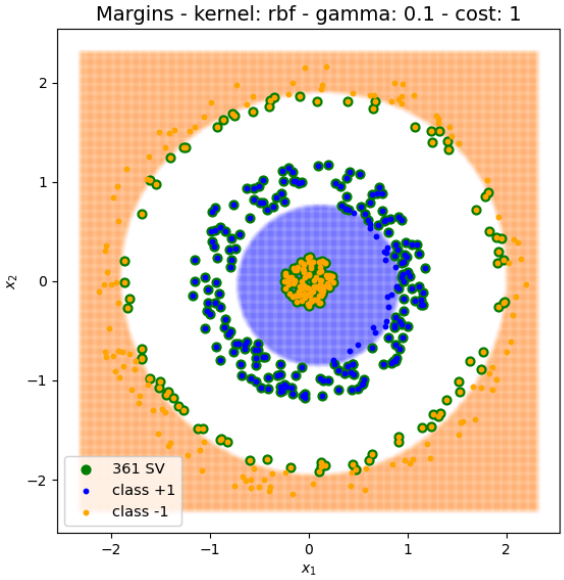
    return models
```

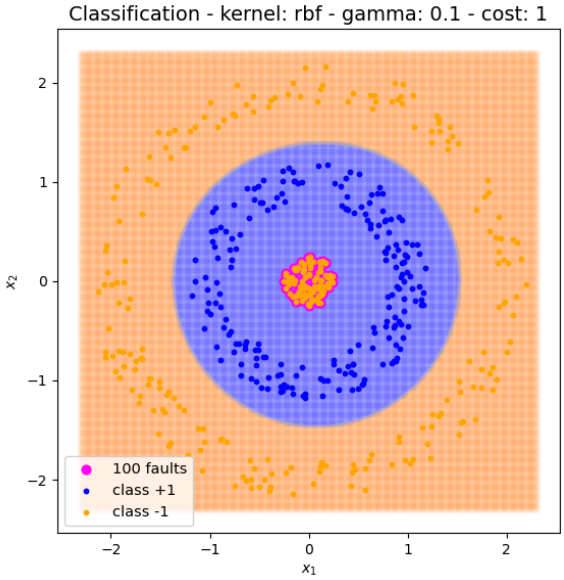
In [27]:
```
# Nothing to do here, just run the cell.
cost_values = [10**i for i in range(-1, 4)]
gamma_values = [0.1, 0.5 ,0.9]

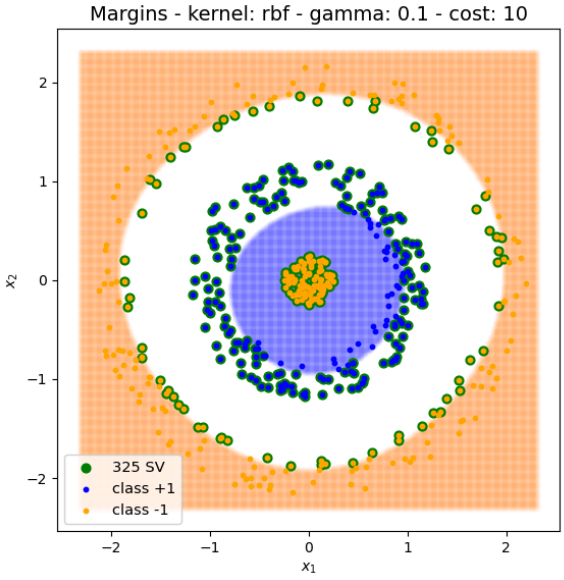
models_gamma_C = iter_gamma_C(gamma_values, cost_values, X_new, y_new)
```

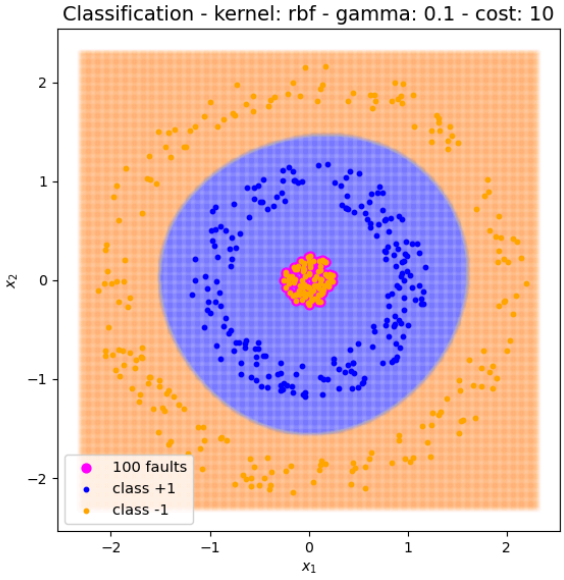


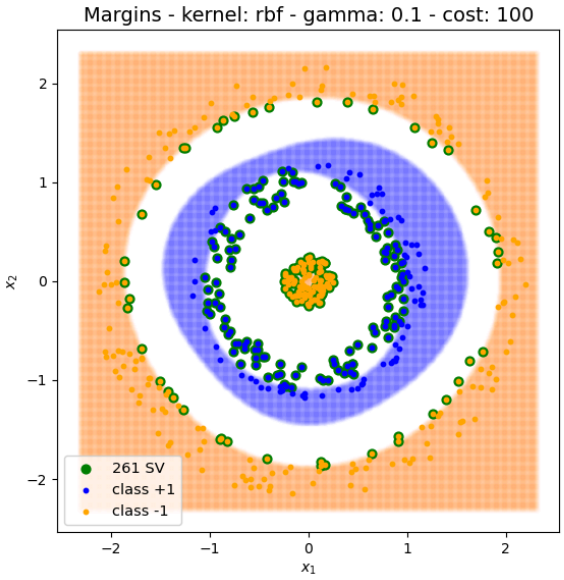


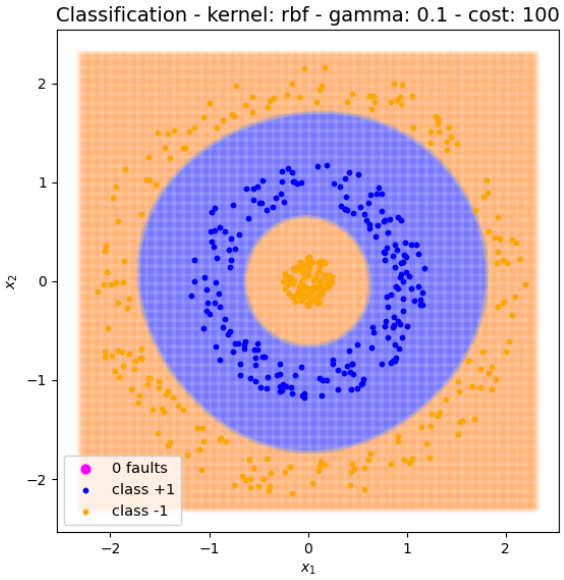


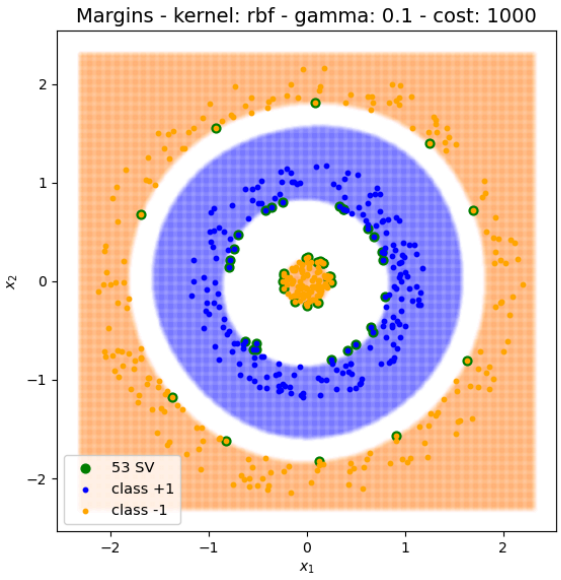


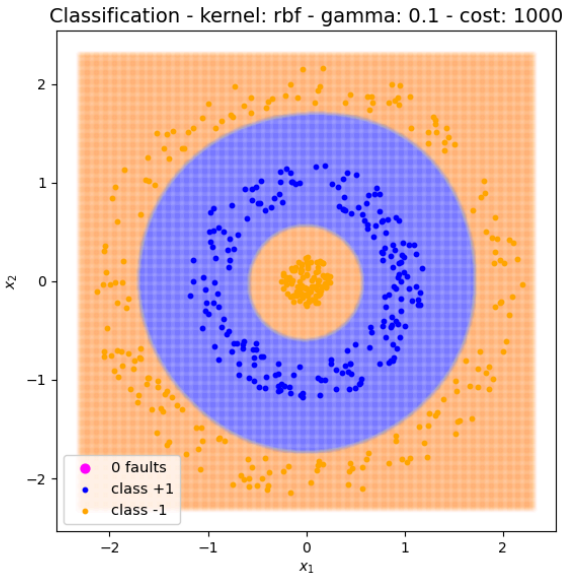

Margins - kernel: rbf - gamma: 0.1 - cost: 10
325 SV
class +1
class -1
x_1
x_2

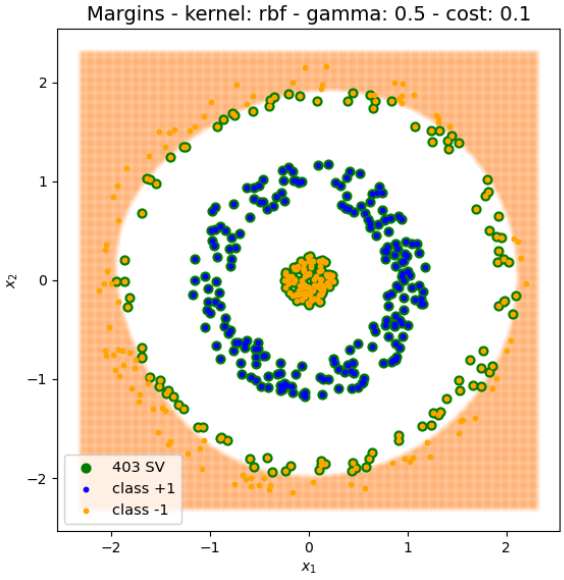

Classification - kernel: rbf - gamma: 0.1 - cost: 10
100 faults
class +1
class -1
x_1
x_2

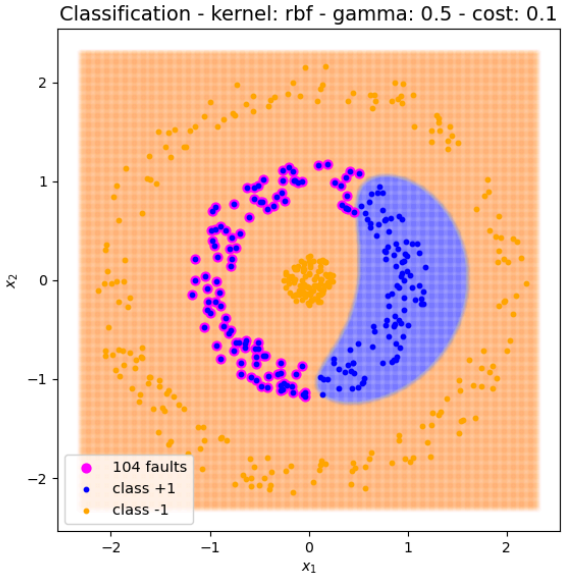

Margins - kernel: rbf - gamma: 0.1 - cost: 100
261 SV
class +1
class -1
x_1
x_2

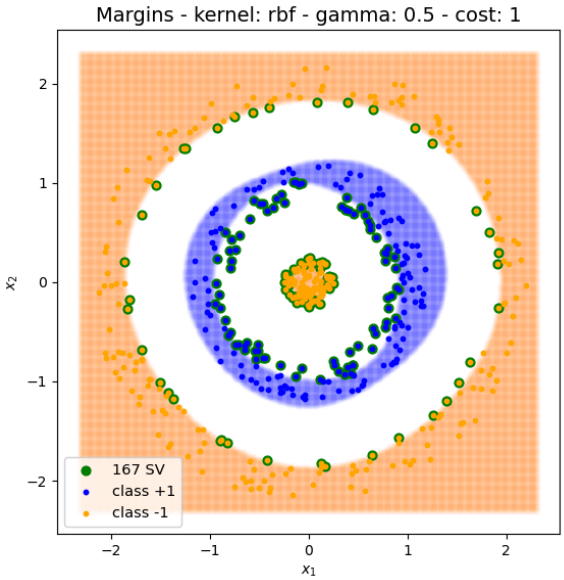

Classification - kernel: rbf - gamma: 0.1 - cost: 100
0 faults
class +1
class -1
x_1
x_2

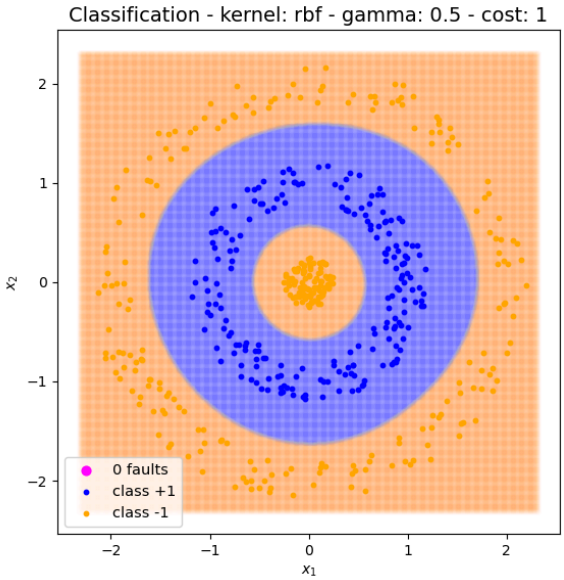

Margins - kernel: rbf - gamma: 0.1 - cost: 1000
53 SV
class +1
class -1
x_1
x_2

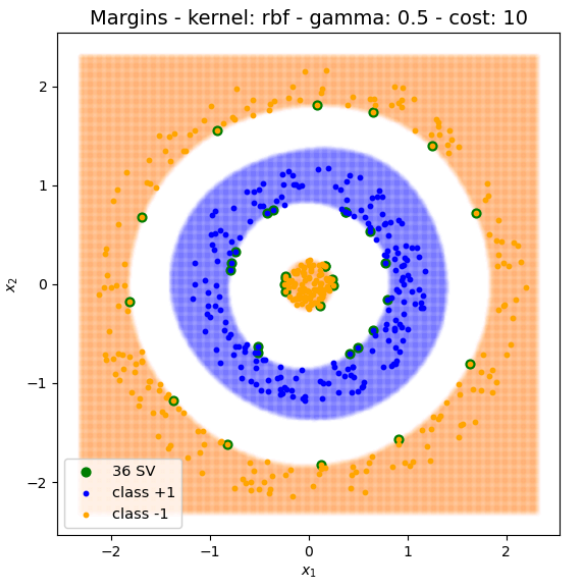

Classification - kernel: rbf - gamma: 0.1 - cost: 1000
0 faults
class +1
class -1
x_1
x_2

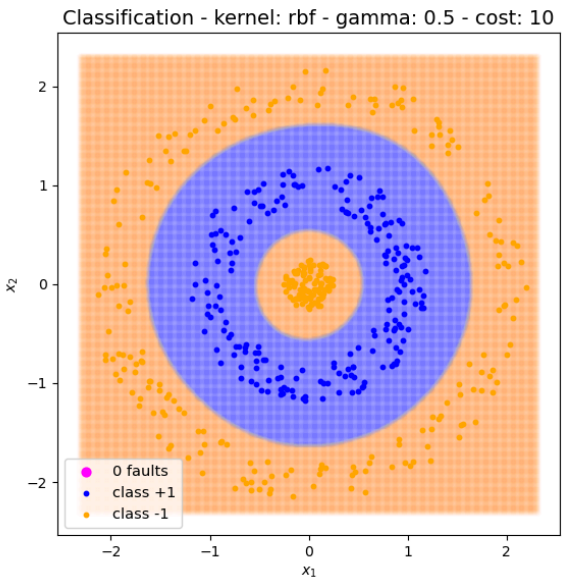

Margins - kernel: rbf - gamma: 0.5 - cost: 0.1
403 SV
class +1
class -1
x_1
x_2

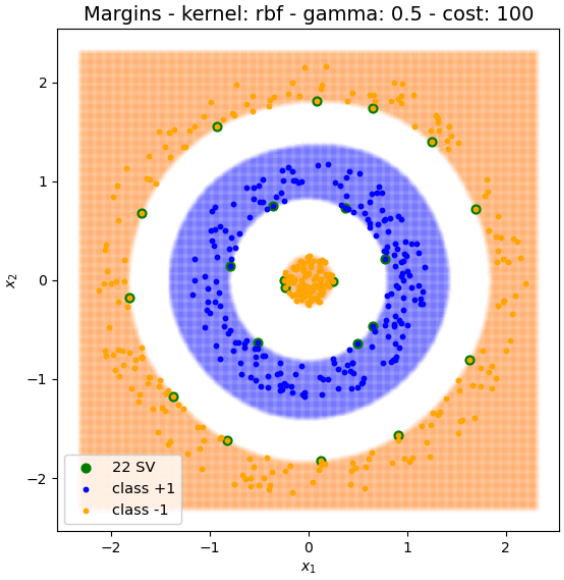

Classification - kernel: rbf - gamma: 0.5 - cost: 0.1
104 faults
class +1
class -1
x_1
x_2

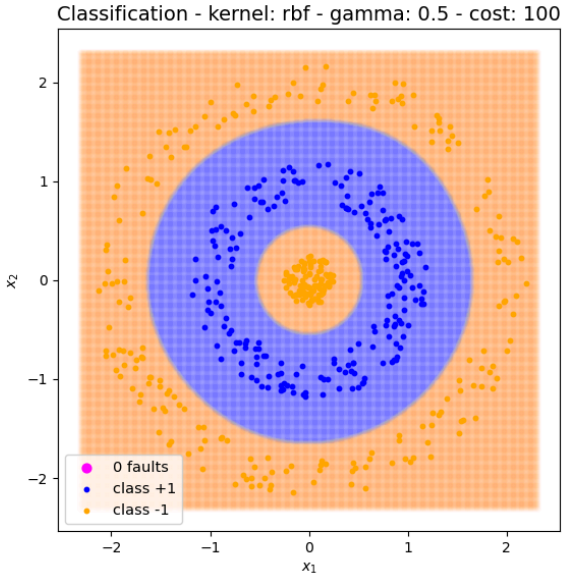

Margins - kernel: rbf - gamma: 0.5 - cost: 1
167 SV
class +1
class -1
x_1
x_2

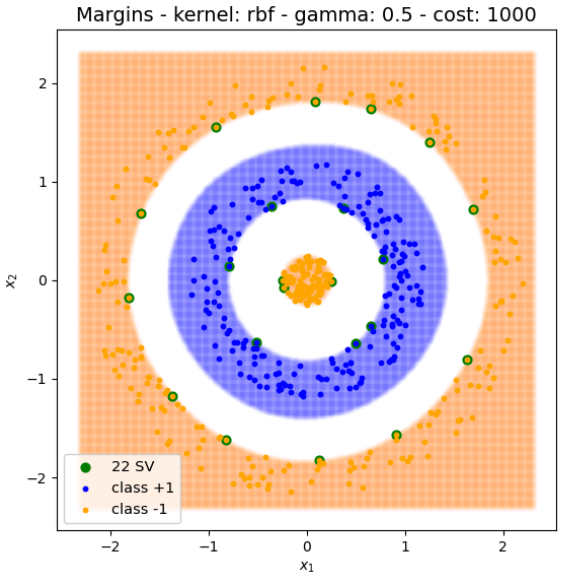

Classification - kernel: rbf - gamma: 0.5 - cost: 1
0 faults
class +1
class -1
x_1
x_2

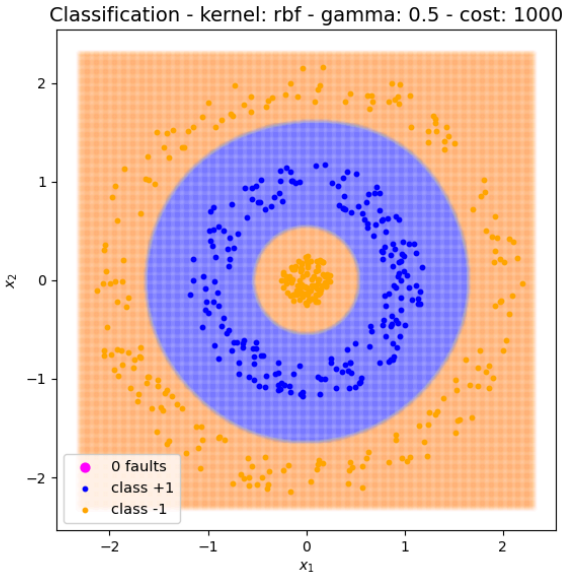

Margins - kernel: rbf - gamma: 0.5 - cost: 10
36 SV
class +1
class -1
x_1
x_2

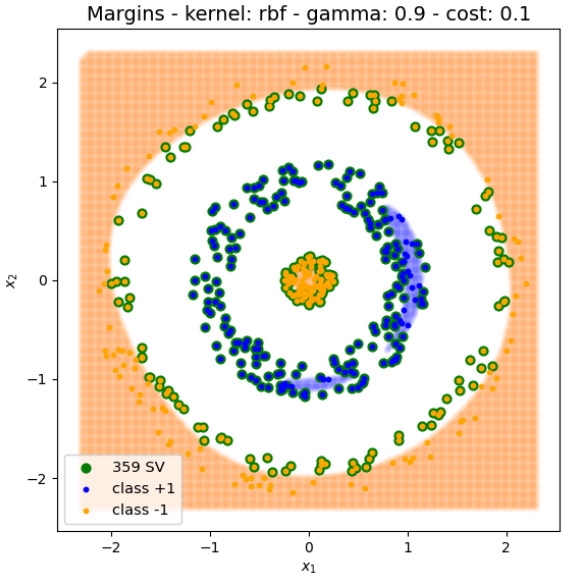

Classification - kernel: rbf - gamma: 0.5 - cost: 10
0 faults
class +1
class -1
x_1
x_2

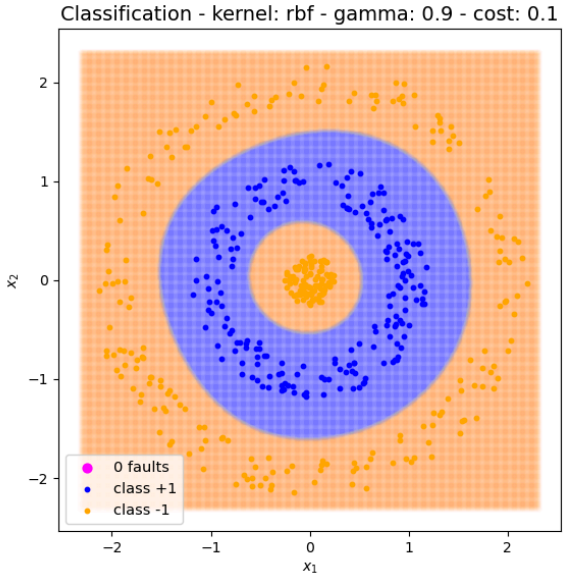

Margins - kernel: rbf - gamma: 0.5 - cost: 100
22 SV
class +1
class -1
x_1
x_2

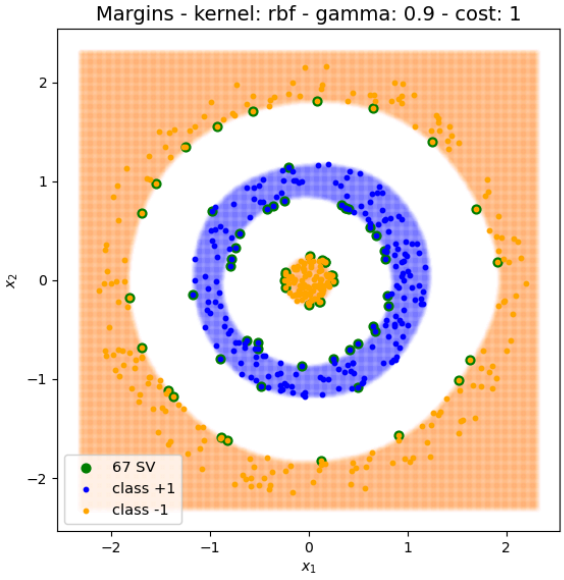

Classification - kernel: rbf - gamma: 0.5 - cost: 100
0 faults
class +1
class -1
x_1
x_2

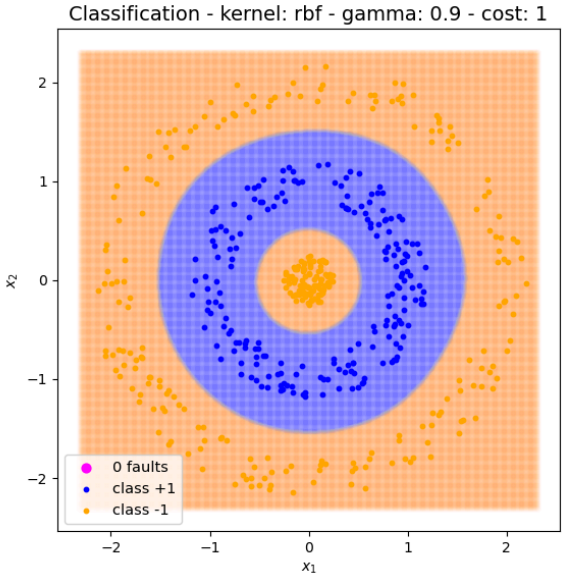

Margins - kernel: rbf - gamma: 0.5 - cost: 1000
22 SV
class +1
class -1
x_1
x_2

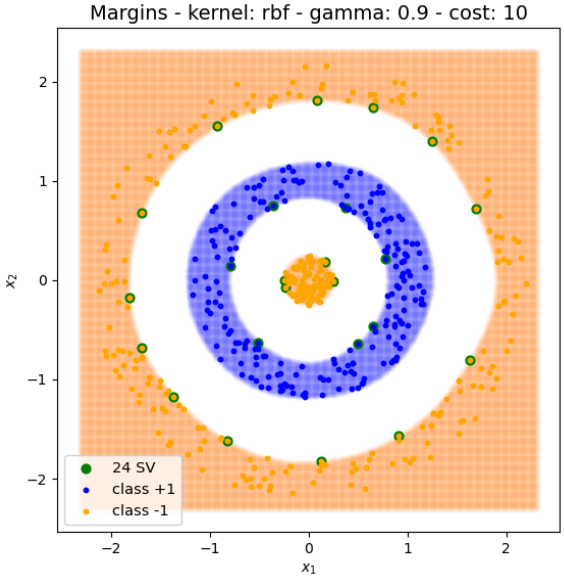

Classification - kernel: rbf - gamma: 0.5 - cost: 1000
0 faults
class +1
class -1
x_1
x_2

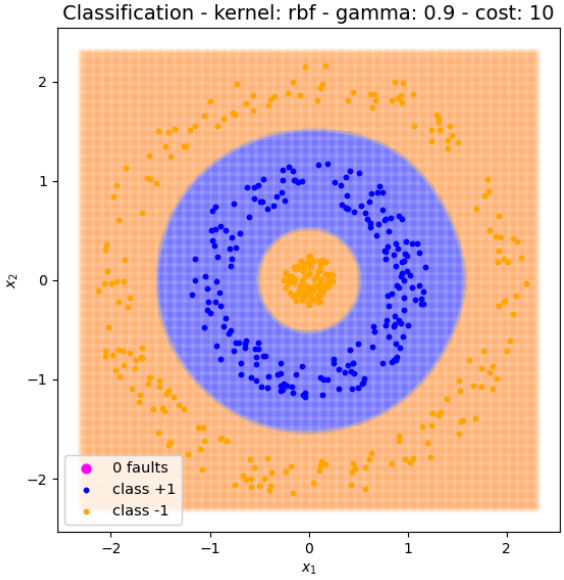

Margins - kernel: rbf - gamma: 0.9 - cost: 0.1
359 SV
class +1
class -1
x_1
x_2

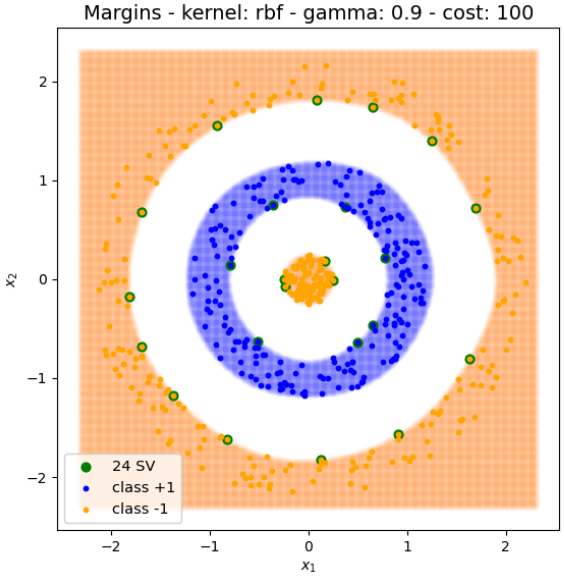

Classification - kernel: rbf - gamma: 0.9 - cost: 0.1
0 faults
class +1
class -1
x_1
x_2

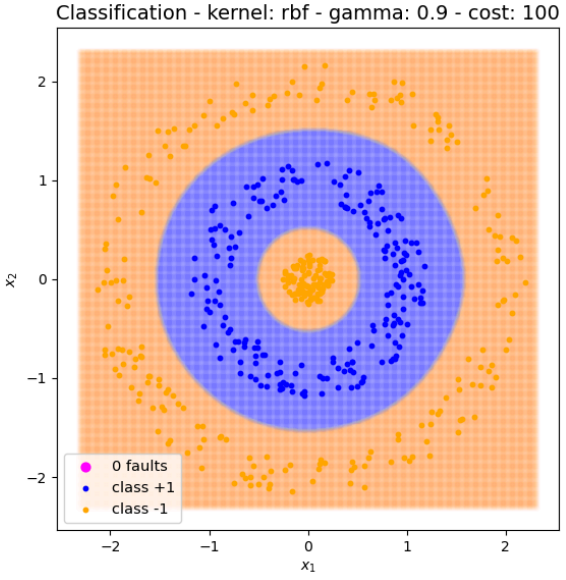

Margins - kernel: rbf - gamma: 0.9 - cost: 1
67 SV
class +1
class -1
x_1
x_2

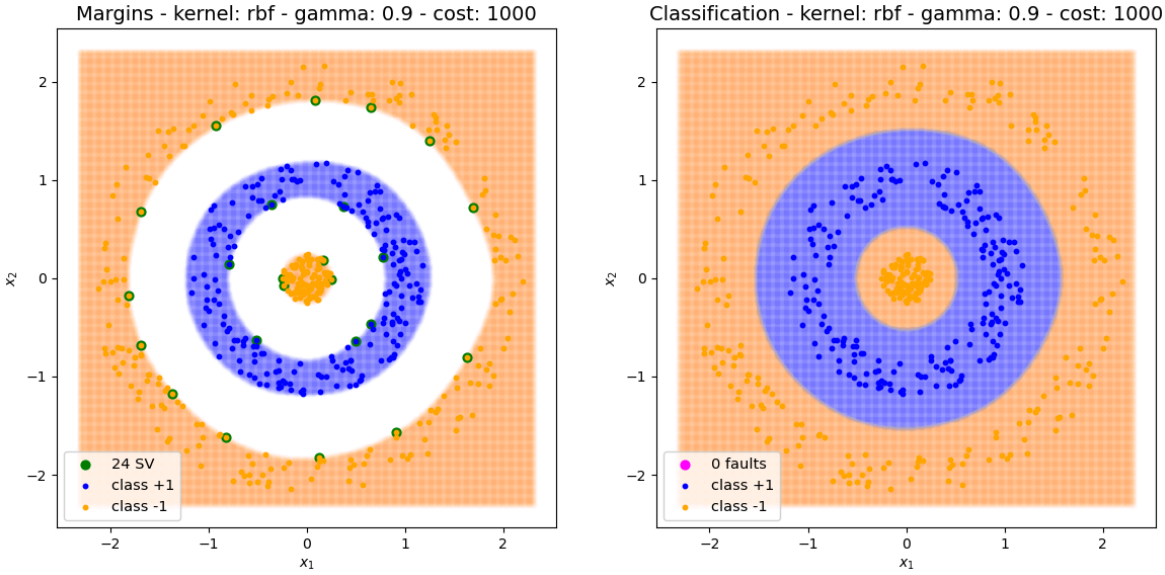

Classification - kernel: rbf - gamma: 0.9 - cost: 1
0 faults
class +1
class -1
x_1
x_2


Margins - kernel: rbf - gamma: 0.9 - cost: 10
24 SV
class +1
class -1
x_1
x_2


Classification - kernel: rbf - gamma: 0.9 - cost: 10
0 faults
class +1
class -1
x_1
x_2


Margins - kernel: rbf - gamma: 0.9 - cost: 100
24 SV
class +1
class -1
x_1
x_2


Classification - kernel: rbf - gamma: 0.9 - cost: 100
0 faults
class +1
class -1
x_1
x_2



In [28]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(models_gamma_C, list), "The output of iter_gamma_C is n
assert len(models_gamma_C) == len(C_values) * len(gamma_values), "Some mo
assert all(isinstance(model, dict) for model in models_gamma_C), "At leas
```

Question 3.2 (5 Points):

What observations can you make from your plots? Evaluate, if the given statements are true of false (several options may be correct):

a3_) For a fixed γ, the larger the cost C, the higher the number of support vectors.
b3_) For relatively large C and relatively large γ (say $C \geq 100$ and $\gamma > 0.5$), enlarging γ further doens't improve your performance significantly.
c3_) For fixed C, increasing γ (i.e. decreasing the width of the Gaussian) typically increases the model complexity of the SVM.
d3_) For a fixed gamma value, the higher the cost, the smaller the margin.

To answer the question, assign `True` or `False` boolean values to variables in the next cell. For example, if you think that **a3_)** is correct, define a variable `a3_` and set it to `True`, the same applies to **b3_)** and the other options. A non-correctly answered question as well as no answer (i.e. answer “None”) yields 0 points for a specific question.

In [29]:
```
# YOUR CODE HERE
a3_ = False
b3_ = True
c3_ = True
d3_ = False
```

In [30]:
```
# DO NOT DELETE THIS CELL!
assert a3_ is not None, "Store True/False!"
assert a3_ in [True, False], "Invalid Answer!"
```

In [31]:
```
# DO NOT DELETE THIS CELL!
assert b3_ is not None, "Store True/False!"
assert b3_ in [True, False], "Invalid Answer!"
```

In [32]:
```
# DO NOT DELETE THIS CELL!
assert c3_ is not None, "Store True/False!"
assert c3_ in [True, False], "Invalid Answer!"
```

In [33]:
```
# DO NOT DELETE THIS CELL!
assert d3_ is not None, "Store True/False!"
assert d3_ in [True, False], "Invalid Answer!"
```

Task 4: SVM with an rbf kernel and an outlier

Finally, we want to investigate, how the rbf kernel classifier reacts to outliers.

- **Code 4.1**:
  - To this end, implement the function `add_point` that adds a single (outlier) point to your new data set `X_new`, `y_new` (i.e. the data with the additional disk with -1 labels) and label this point with $y = +1$.
  - We are going to use an rbf kernel and play around with the parameter to explore the effects on the classification performance by again appropriately using the `iter_gamma_C` function. We are trying out small costs $C \sim 0.1$ and large costs $C \sim 1000$ and also iterate over different values of γ, ranging from 0.1 to 1.
- **Question 4.1**:
  - One last time, answer some questions about the resulting plots.

Code 4.1 (5 Points):

In [34]:
```
def add_point(X: np.ndarray, y: np.ndarray, data_point: np.ndarray) -> tu
    """ Add a new data point to the input arrays.

    This function appends a new `data_point` to the feature array `X`
    and a label `1` to the target array `y`.

    Parameters
    ----------
    X : (N, 2) np.ndarray
        The data matrix containing the original data points.
    y : (N,) np.ndarray
        The target vector containing the original labels.
    data_point : (1, 2) np.ndarray
        The new datapoint to add.

    Returns
    -------
    tuple of np.ndarray
        - X_new : (N+1, 2) np.ndarray
            The updated data matrix with the new point added.
        - y_new : (N+1,) np.ndarray
```

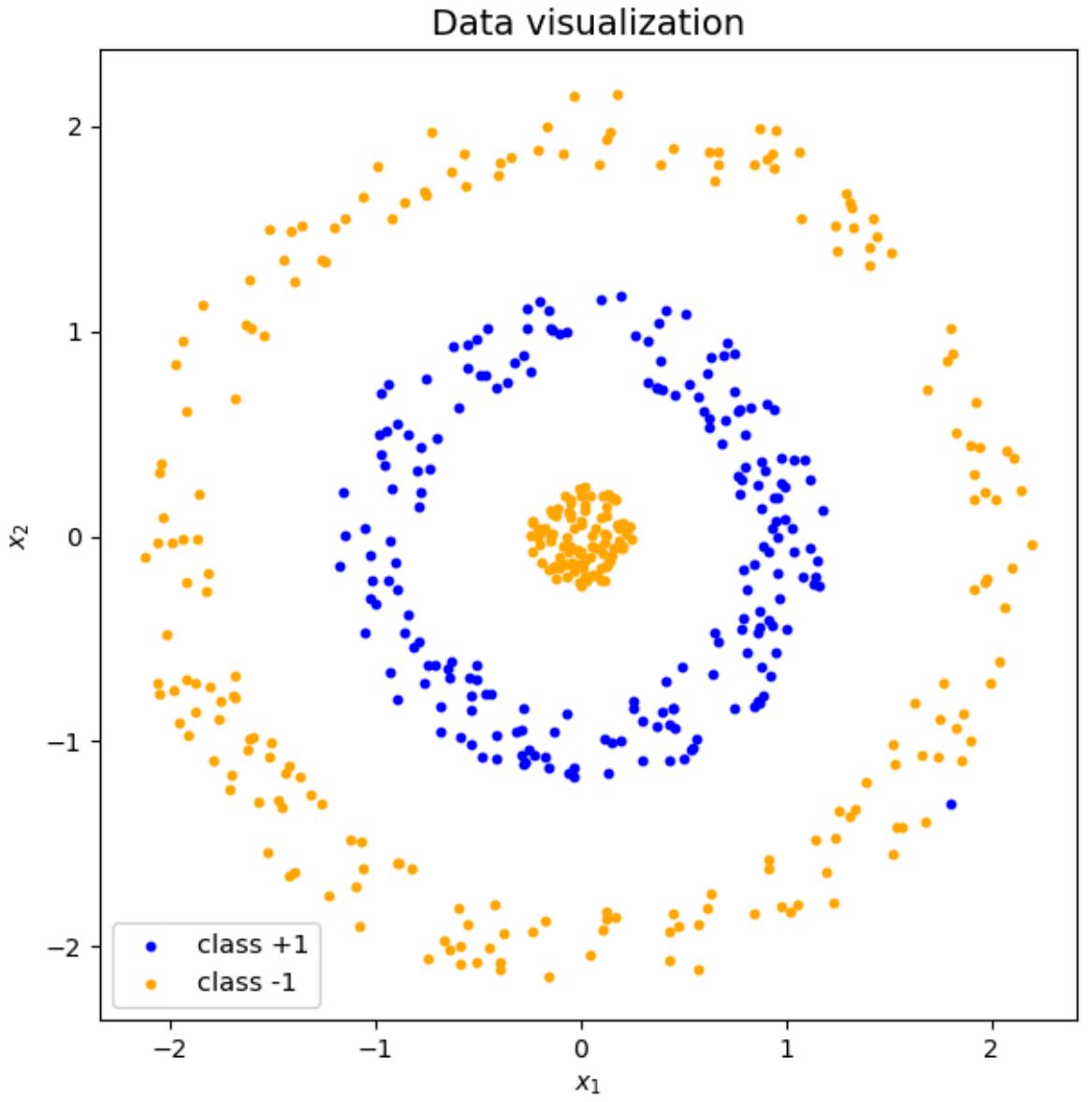
```
            The updated target vector with the new label added.
    """
    # YOUR CODE HERE
    X_new = np.vstack((X, data_point))
    y_new = np.hstack((y, 1))

    return X_new, y_new
```

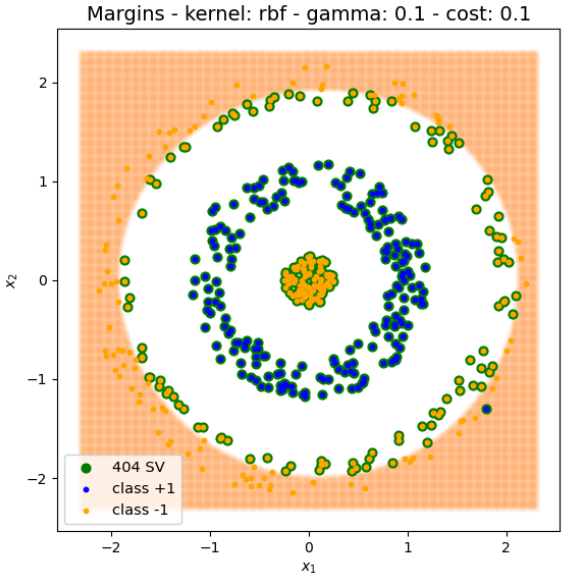
In [35]:
```
outlier = np.array([[1.8, -1.3]])
X_with_outlier, y_with_outlier = add_point(X_new, y_new, outlier)

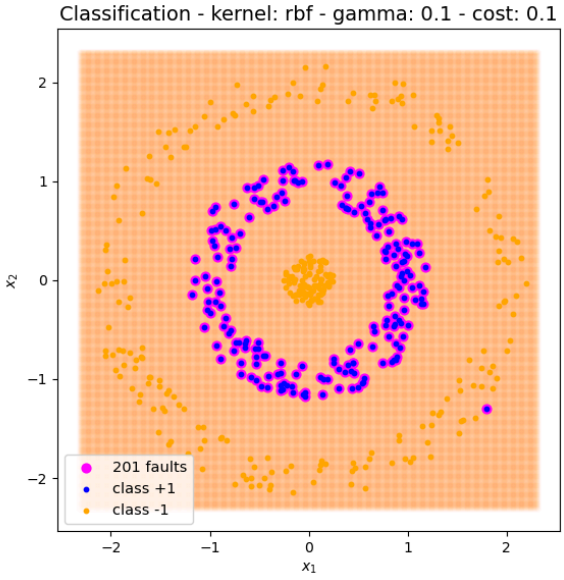
assert isinstance(X_with_outlier, np.ndarray) and isinstance(y_with_outli
assert X_with_outlier.shape == (X_new.shape[0]+1, X_new.shape[1]), "You s
np.testing.assert_array_equal(X_with_outlier[-1:], outlier)
assert y_with_outlier[-1] == 1, "The label of the outlier should be +1!"
```

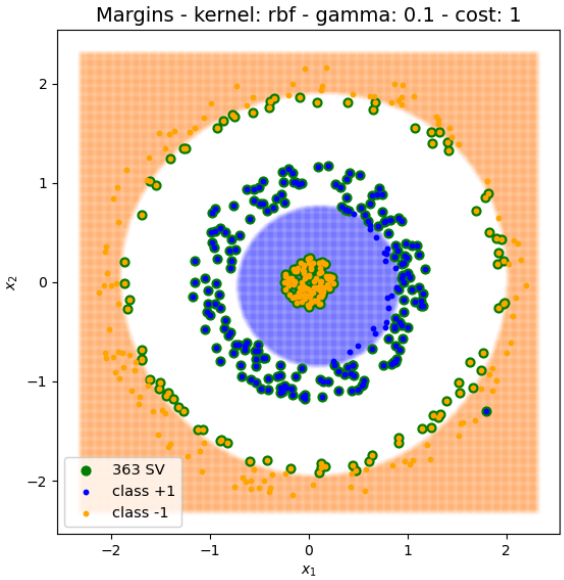
In [36]:
```
plot_data(X_with_outlier, y_with_outlier)
```

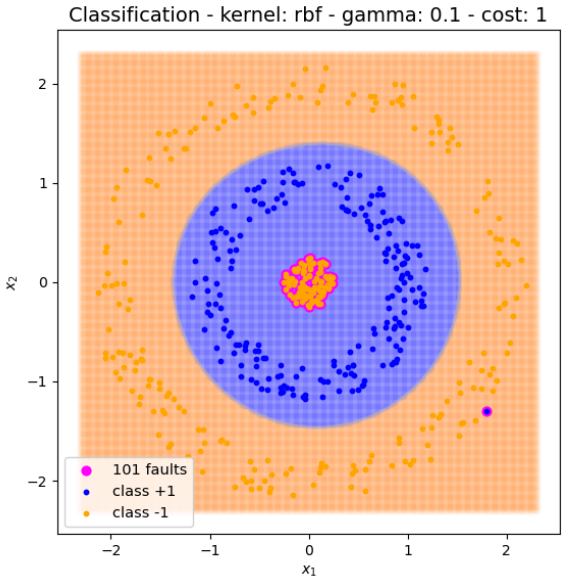


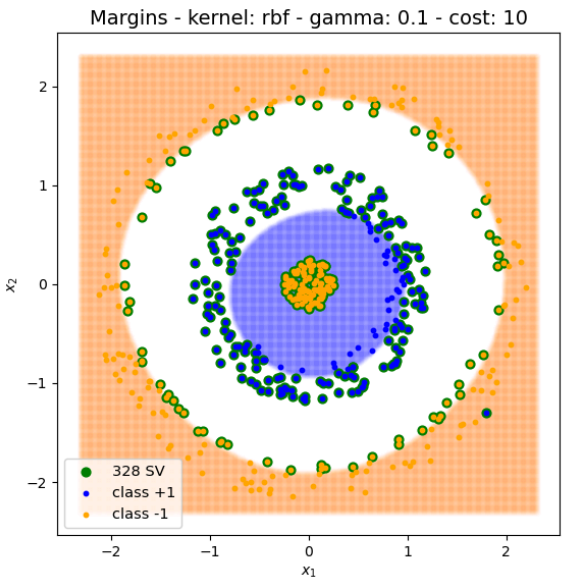
In [37]:
```
_ = iter_gamma_C(gamma_values, cost_values, X_with_outlier, y_with_outlie
```

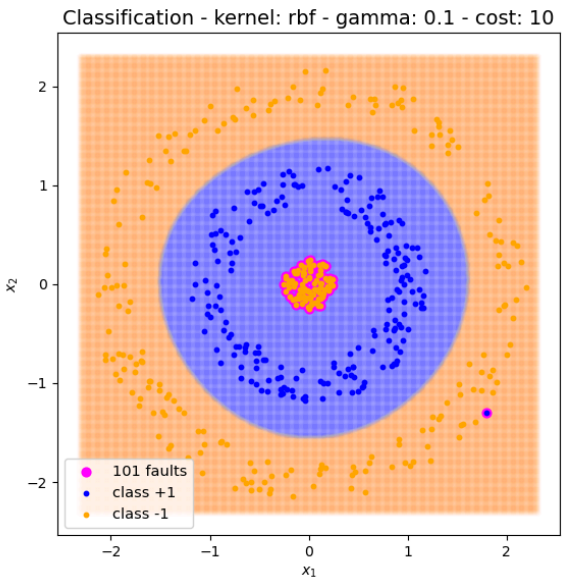

Margins - kernel: rbf - gamma: 0.1 - cost: 0.1
404 SV
class +1
class -1
x_1
x_2

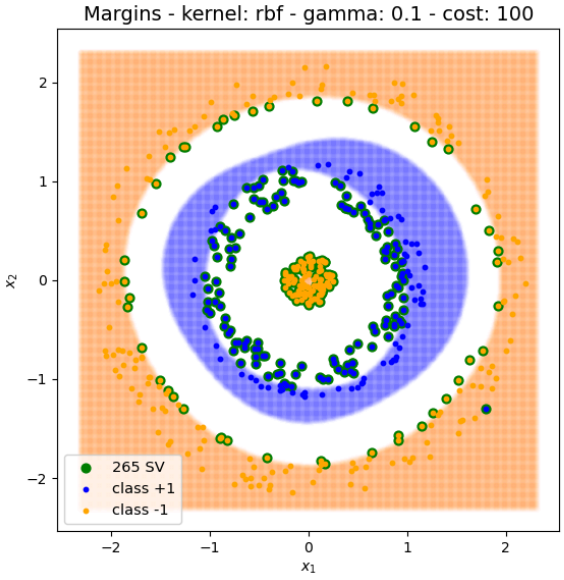

Classification - kernel: rbf - gamma: 0.1 - cost: 0.1
201 faults
class +1
class -1
x_1
x_2

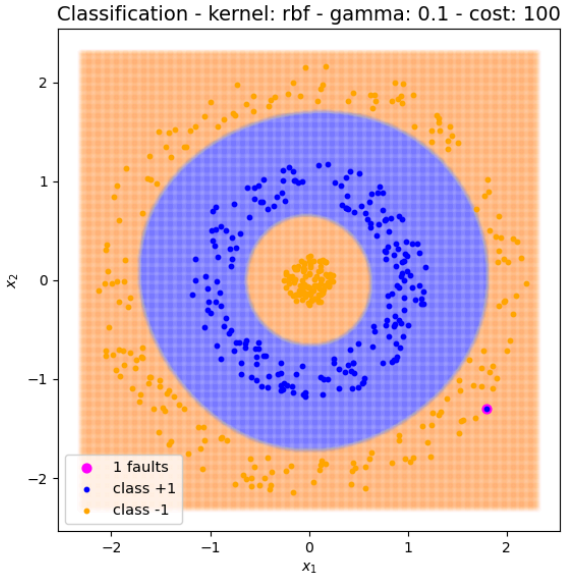

Margins - kernel: rbf - gamma: 0.1 - cost: 1
363 SV
class +1
class -1
x_1
x_2

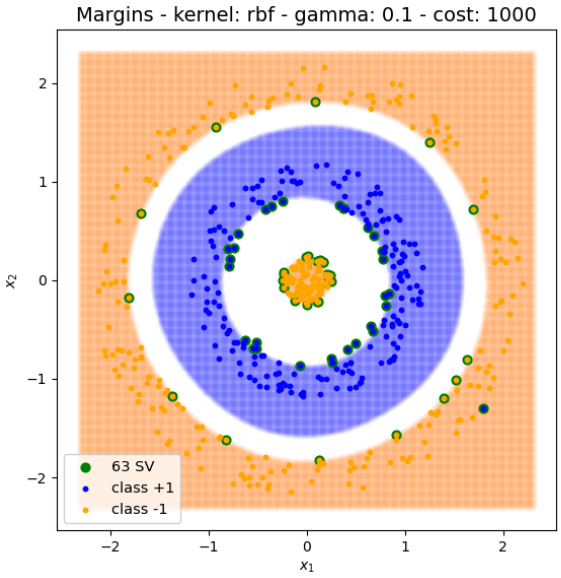

Classification - kernel: rbf - gamma: 0.1 - cost: 1
101 faults
class +1
class -1
x_1
x_2

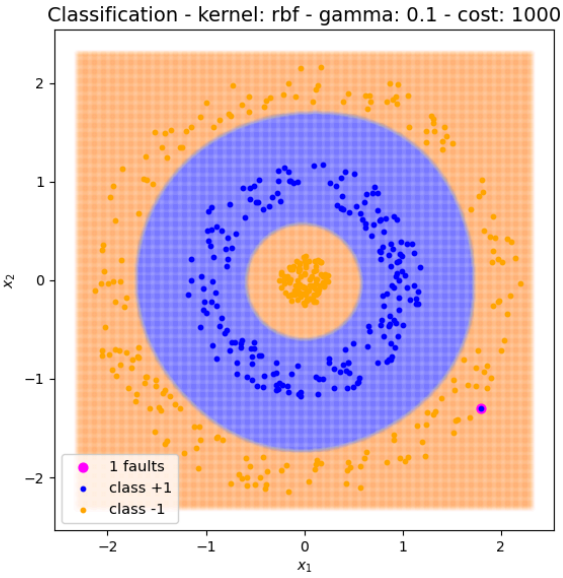

Margins - kernel: rbf - gamma: 0.1 - cost: 10
328 SV
class +1
class -1
x_1
x_2

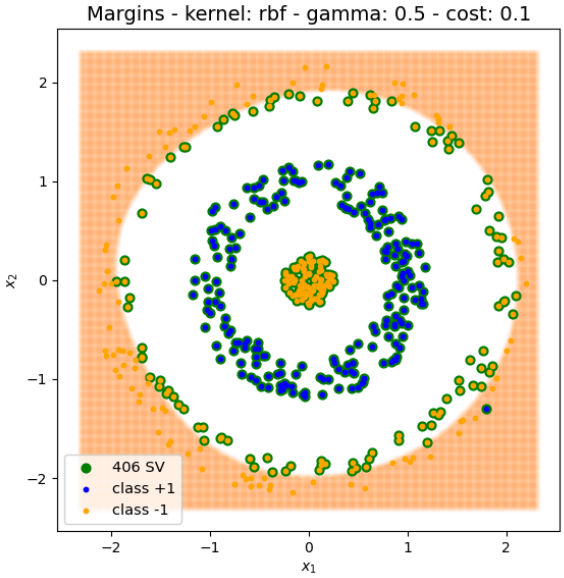

Classification - kernel: rbf - gamma: 0.1 - cost: 10
101 faults
class +1
class -1
x_1
x_2

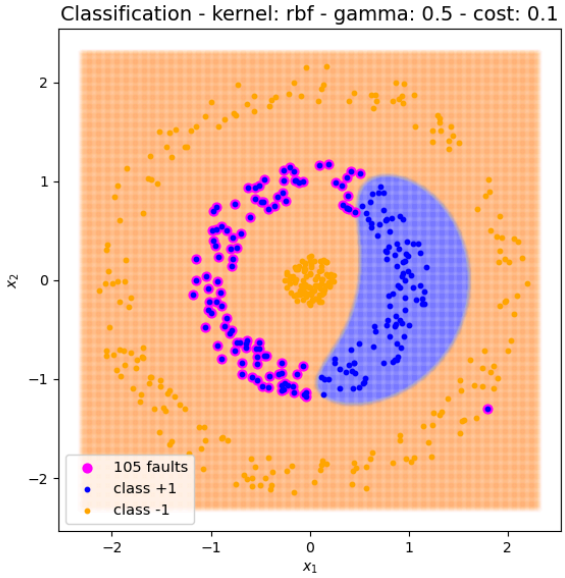

Margins - kernel: rbf - gamma: 0.1 - cost: 100
265 SV
class +1
class -1
x_1
x_2

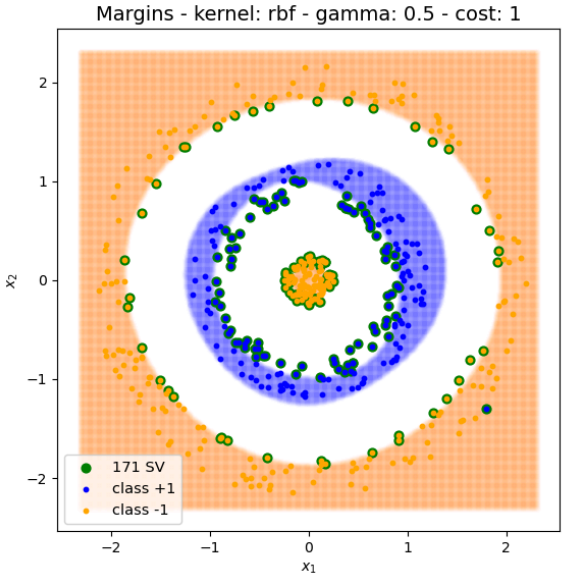

Classification - kernel: rbf - gamma: 0.1 - cost: 100
1 faults
class +1
class -1
x_1
x_2

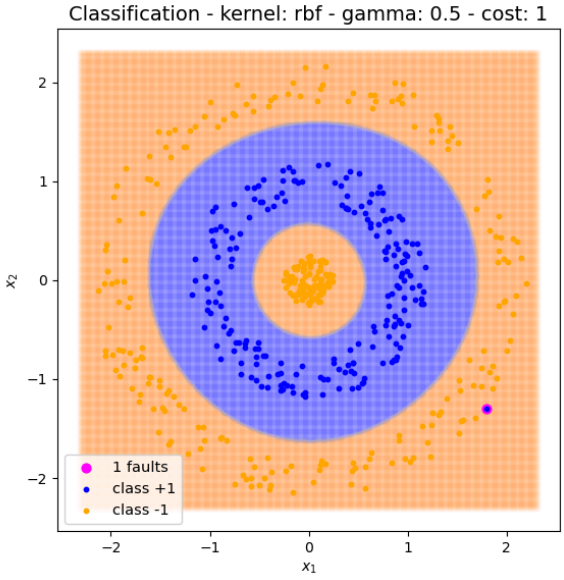

Margins - kernel: rbf - gamma: 0.1 - cost: 1000
63 SV
class +1
class -1
x_1
x_2

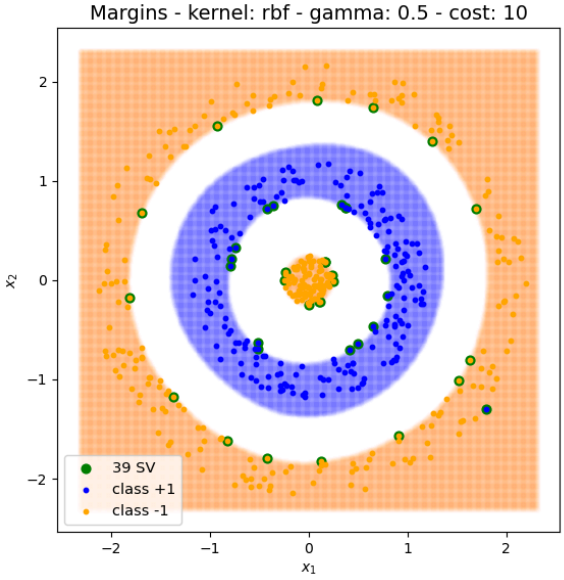

Classification - kernel: rbf - gamma: 0.1 - cost: 1000
1 faults
class +1
class -1
x_1
x_2

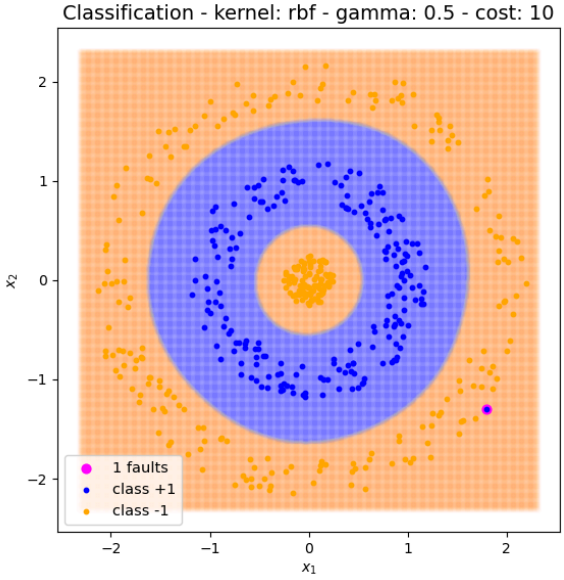

Margins - kernel: rbf - gamma: 0.5 - cost: 0.1
406 SV
class +1
class -1
x_1
x_2

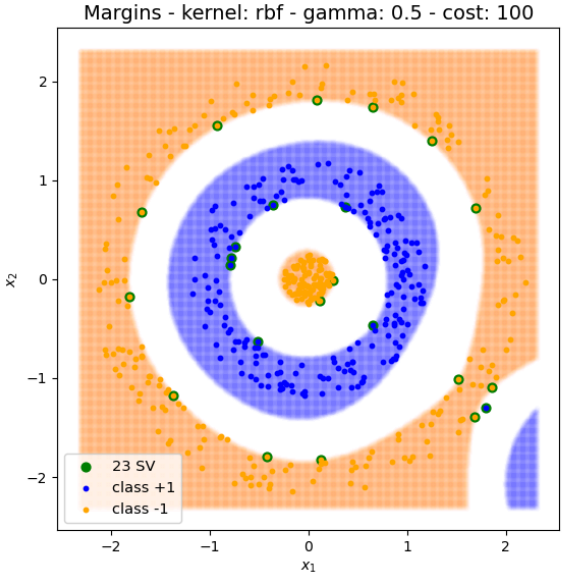

Classification - kernel: rbf - gamma: 0.5 - cost: 0.1
105 faults
class +1
class -1
x_1
x_2

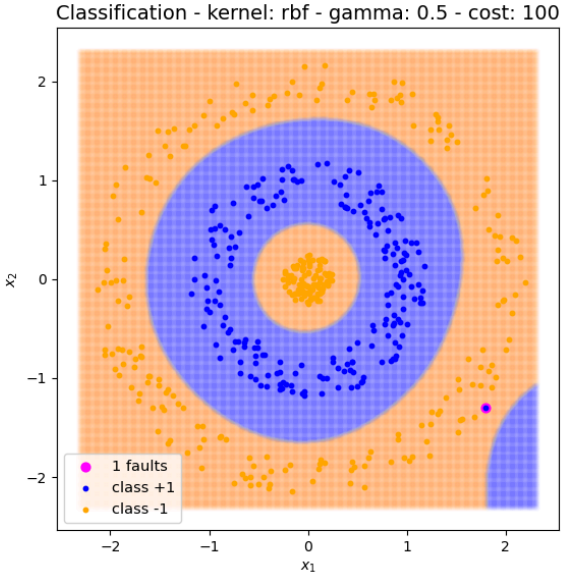

Margins - kernel: rbf - gamma: 0.5 - cost: 1
171 SV
class +1
class -1
x_1
x_2

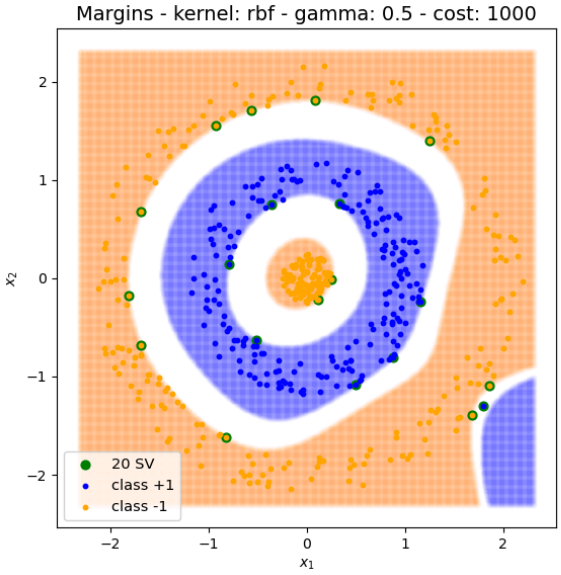

Classification - kernel: rbf - gamma: 0.5 - cost: 1
1 faults
class +1
class -1
x_1
x_2

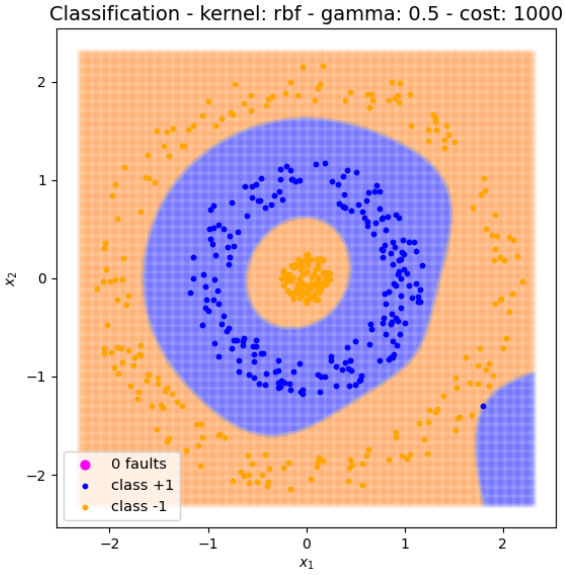

Margins - kernel: rbf - gamma: 0.5 - cost: 10
39 SV
class +1
class -1
x_1
x_2

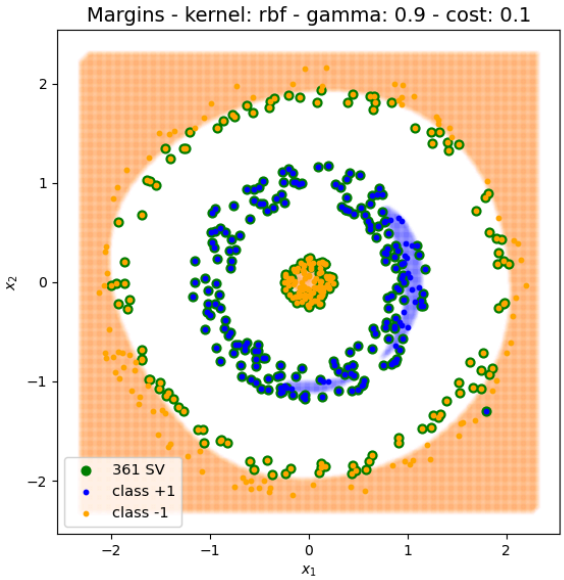

Classification - kernel: rbf - gamma: 0.5 - cost: 10
1 faults
class +1
class -1
x_1
x_2

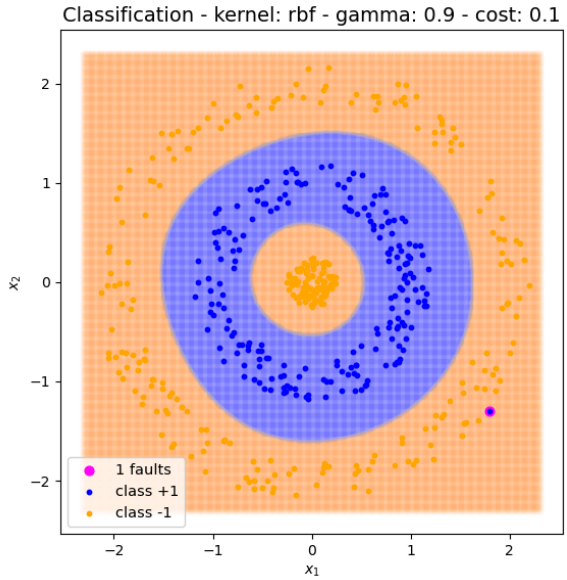

Margins - kernel: rbf - gamma: 0.5 - cost: 100
23 SV
class +1
class -1
x_1
x_2

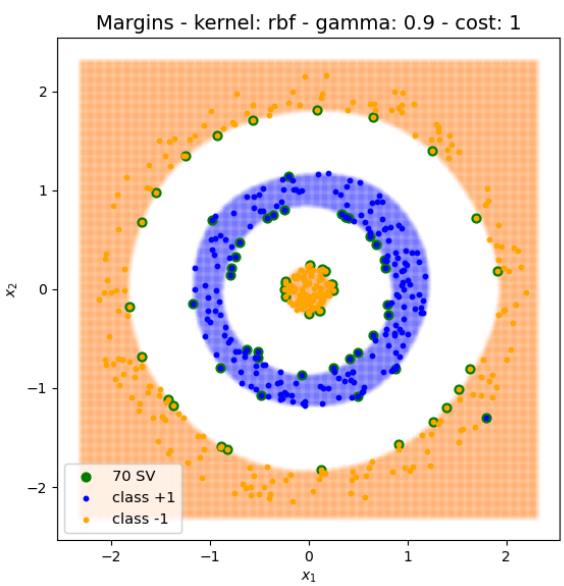

Classification - kernel: rbf - gamma: 0.5 - cost: 100
1 faults
class +1
class -1
x_1
x_2

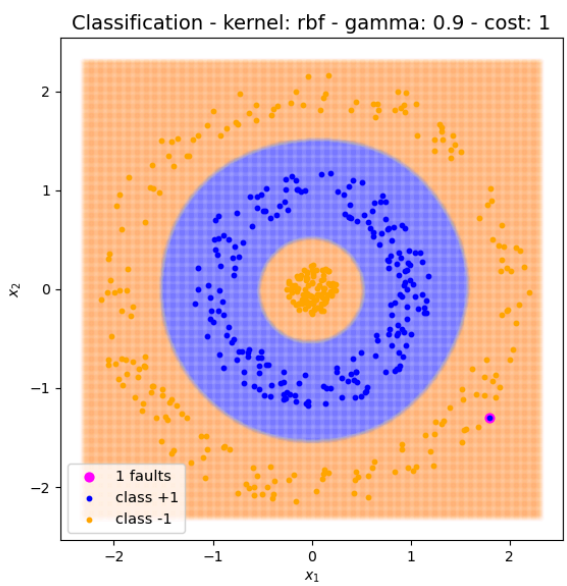

Margins - kernel: rbf - gamma: 0.5 - cost: 1000
20 SV
class +1
class -1
x_1
x_2

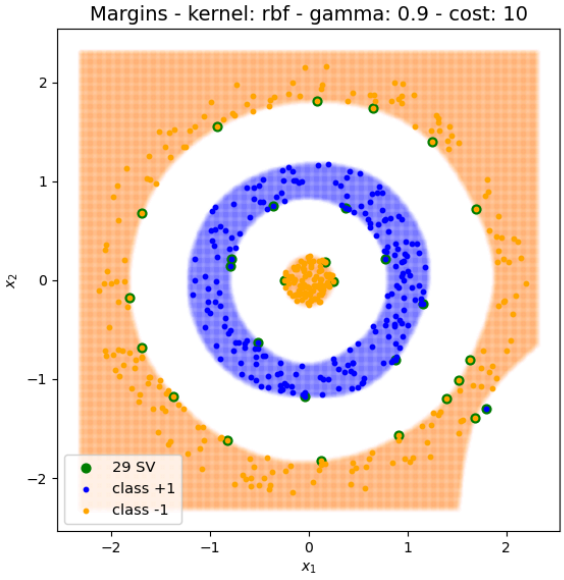

Classification - kernel: rbf - gamma: 0.5 - cost: 1000
0 faults
class +1
class -1
x_1
x_2

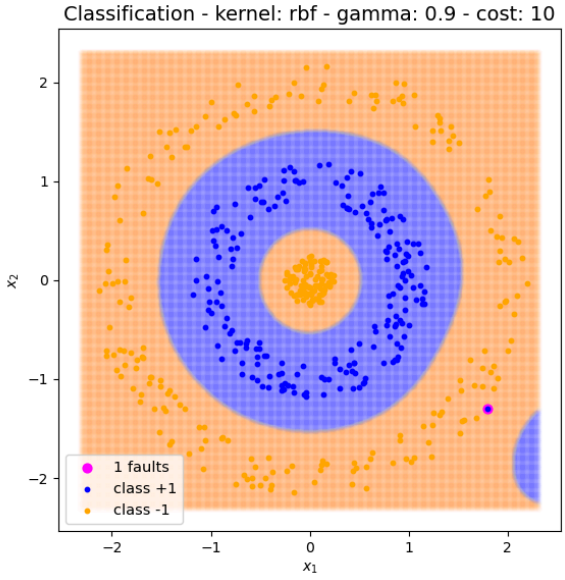

Margins - kernel: rbf - gamma: 0.9 - cost: 0.1
361 SV
class +1
class -1
x_1
x_2

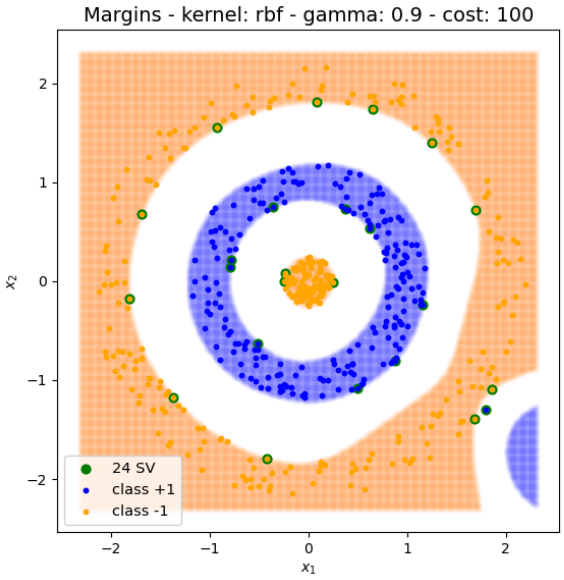

Classification - kernel: rbf - gamma: 0.9 - cost: 0.1
1 faults
class +1
class -1
x_1
x_2

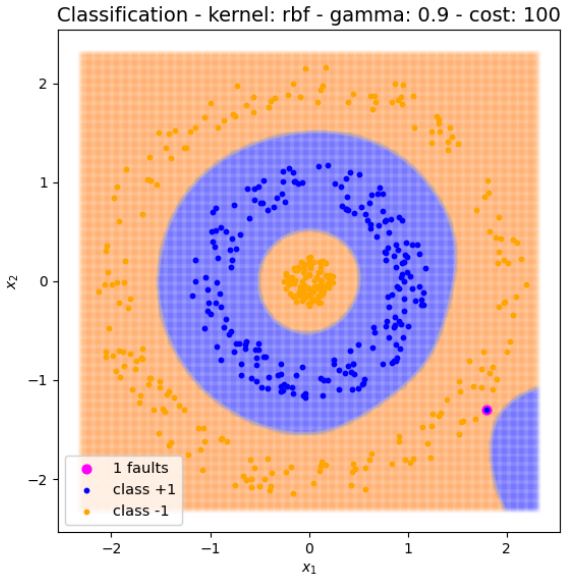

Margins - kernel: rbf - gamma: 0.9 - cost: 1
70 SV
class +1
class -1
x_1
x_2

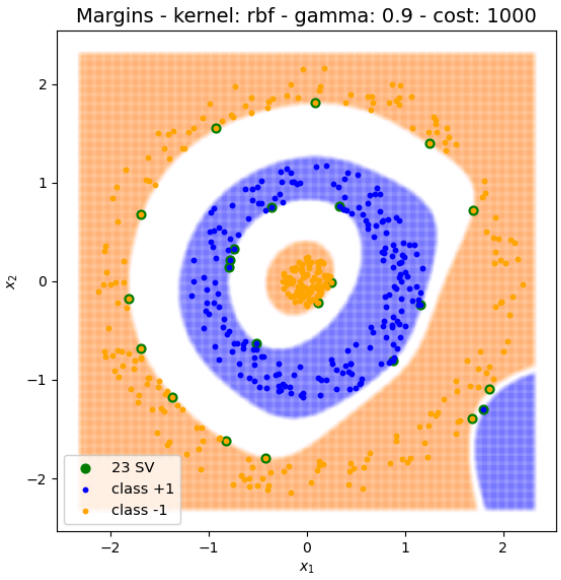

Classification - kernel: rbf - gamma: 0.9 - cost: 1
1 faults
class +1
class -1
x_1
x_2

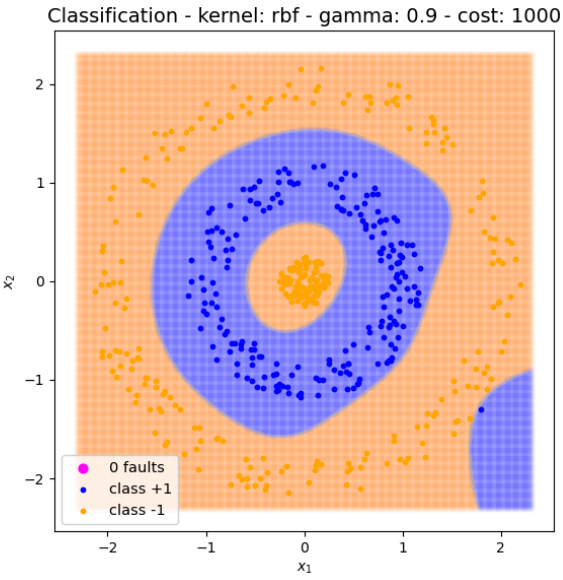












Question 4.1 (5 Points):

What observations can you make from your plots? Tick the correct boxes:

a4_) For relatively low costs (e.g. $C \leq 1$) and an appropriately chosen γ (e.g. 0.9) the classifier correctly classifies all points except for the outlier.
b4_) For relatively high costs (e.g. $C \geq 100$) the classifier always, i.e. independent of γ, shows a region of the positive class near the outlier.
c4_) Classifiers with high costs ($C \geq 100$ say) and high γ ($\gamma \geq 0.5$) are susceptible to overfitting.
d4_) For relatively high costs ($C \geq 100$) and small γ (0.1), the SVM correctly classifies except for the outlier.

To answer the question, assign `True` or `False` boolean values to variables in the next cell. For example, if you think that **a4_)** is correct, define a variable `a4_` and set it to `True`, the same applies to **b4_)** and the other options. A non-correctly answered question as well as no answer (i.e. answer "None") yields 0 points for a specific question.

In [38]:
```
# YOUR CODE HERE
a4_ = False
b4_ = False
c4_ = True
d4_ = True
```

In [39]:
```
# DO NOT DELETE THIS CELL!
assert a4_ is not None, "Store True/False!"
assert a4_ in [True, False], "Invalid Answer!"
```

In [44]:
```
# DO NOT DELETE THIS CELL!
assert b4_ is not None, "Store True/False!"
assert b4_ in [True, False], "Invalid Answer!"
```

In [43]:
```
# DO NOT DELETE THIS CELL!
assert c4_ is not None, "Store True/False!"
assert c4_ in [True, False], "Invalid Answer!"
```

In [42]:
```
# DO NOT DELETE THIS CELL!
assert d4_ is not None, "Store True/False!"
assert d4_ in [True, False], "Invalid Answer!"
```